

ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)
ORGANISATION OF ISLAMIC COOPERATION (OIC)
Department of Computer Science and Engineering (CSE)

Final Project Report

Project Name

Khoj

Submitted By:

Group 12

Talha Zubayer (230042119)
Ayman Rahman Bhuyian (230042140)
Md Nadid Hasan (230042146)

SWE 4404 : SPL II

Supervised By

Yousuf Ibne Kamal Niloy
Junior Lecturer
Department of Computer Science and Engineering

Table of Content

1.Explanation of the Model Diagram **4**
1.1 The problem. 4
1.2 What we built. 4
1.3 The outcome 5
2. Problem Domain and Background **6**
2.1 Domain and users 6
2.2 The problem and why it matters 6
2.3 Fundamental concepts behind the problem 7
2.4 Specialist knowledge 7
3. Literature and Existing Solutions **9**
3.1 Existing ways to find a room 9
3.2 Technical sources 10
3.3 How Khoj builds on and differs from them 10
4. Requirements (SRS Session) **11**
4.1 Stakeholders 11
4.2 Elicitation 11
4.3 Functional requirements 12
4.4 Non-functional requirements 13
4.5 Traceability: requirement to stakeholder need 14
4.6 Contextual and sustainable-development considerations 15
4.7 Feedback in the SRS session and changes made 16
5. System Design (Design Session) **17**
5.1 Design goals 17
5.2 Use cases 18
5.3 Architecture and components 19
5.4 Data flow 21
DFD Level: 0 21
DFD Level: 1 21
5.5 Module and data model 22
5.6 Database schema 24
5.7 Key interactions 25
5.8 Design alternatives 29
A. Overall architecture: layered monolith vs. microservices 29
B. Database: MongoDB vs. PostgreSQL 29

C. Media storage: upload after approval vs. upload on submission 31
D. Real-time chat: Socket.IO vs. HTTP polling 31
E. Owner identity: Khatian number vs. e-mail account 32
5.9 Feedback in the Design session and changes made 32
6. Implementation (Implementation Session) 33
6.1 (a) Project structure 33
6.2 (b) Communication between components 36
6.3 (c) Division of modules and methods 38
Rationale: 38
Lines of code 39
Justification of the maximums 40
6.4 Implemented features 40
6.5 Fundamental concepts in the code 41
6.6 Challenges and how they were solved 42
6.7 Following and departing from the design 43
6.8 Refactoring after the Implementation session 44
6.9 User interface 45
Screenshots 46
7. Tools, Version Control, Testing, and CI/CD 57
7.1 Frameworks, libraries and tools (CO4) 57
7.2 Version control and workflow (CO4) 58
Feature milestones in the history: 58
7.3 Testing 59
7.4 CI/CD 60
8. Professional, Ethical, and Legal Considerations 61
8.1 Licences 61
8.2 User data and privacy 62
8.3 Security measures and known gaps 62
Known gaps 63
8.4 Ethical and legal issues in deployment 63
9. Project Management and Feasibility 64
9.1 Process 64
9.2 Planned vs. actual milestones 64
9.3 Scope and schedule changes 66
9.4 Effort, infrastructure and cost estimates 67
9.5 Feasibility 67
10. Teamwork and Individual Contributions 68
10.1 Member Responsibilities 68

10.2 Coordinating interdependent modules 68
10.3 Decisions, blockers and inclusion 69
11. Results, Limitations, and Future Work 70
11.1 Status of each requirement 70
11.2 Non-functional requirements met 73
11.3 Known limitations 74
11.4 Future work 74
12. Glossary and References 75
12.1 Glossary 75
12.2 References 76

1.Explanation of the Model Diagram

1.1 The problem.

Students of the Islamic University of Technology (IUT) who need a room near campus look for one through Facebook groups, "To-Let" signs on buildings, brokers and friends. Nobody checks whether the person offering the room really owns it. A student cannot see the room before going there, cannot compare rents easily, and cannot hear from people who lived there before. New students, who do not know the area, often pay more than others for similar rooms.

1.2 What we built.

Khoj ("search" in Bangla) is a website where IUT students and the homeowners of Board Bazar, the area next to the university, meet in one place.

- Students join with their IUT e-mail address. A student who has just been admitted and does not have that address yet can join with their admission rank.
- A homeowner joins by uploading the land record of the house (the *Khatian* certificate) and a photo of it. An administrator checks the documents before the owner can post anything.
- Every room advertisement is also checked by the administrator before students can see it.
- Students filter rooms by rent, by facilities such as gas, lift or Wi-Fi, and by type (a whole room from an owner, or a seat shared with a student looking for a roommate), see the rooms as pins on a map, and read the house rules, including the time the main gate closes at night.
- Students message the owner inside the website, so neither side has to share a phone number first.
- When a room is rented, the owner names the student who took it. Only that student can later rate the room (on six points, such as cleanliness and safety) and the owner. Future students read these ratings.

1.3 The outcome

Khoj works end to end on a local computer, from sign-up to review, for all three kinds of user: student, homeowner and administrator. It was demonstrated live in the final presentation. Of the 28 user stories, 17 are complete, 6 are partly complete and 5 were left out to keep the scope realistic (Section 11). The website works on phones and computers and has a dark mode. Paying rent through mobile banking, putting the website online for public use, and opening it to other universities are the next steps.

> **(non-technical audience).** This section avoids technical terms. The words that cannot be avoided (Khatian, admission rank) are explained where they first appear and in the glossary (Section 12)

2. Problem Domain and Background

2.1 Domain and users

The domain is the informal student rental market around IUT in Board Bazar, Gazipur. Most students who do not get a seat in the IUT halls rent a room or a seat in a flat nearby, often shared with batchmates. Rooms change hands at the start and end of semesters, when students graduate, move in with friends, or get a hall seat.

User	Need
Current IUT Student	Find a suitable, fairly priced room quickly, and trust the owner and the listing
Newly admitted student	Find a room before classes start, before receiving an IUT e-mail
Departing tenant or student seeking a roommate	Pass the room on, or fill a free seat
Homeowner	Fill empty rooms between semesters with verified students
Administrator	Keep fake owners and listings off the platform

2.2 The problem and why it matters

Interviews with three students and one homeowner (Section 4.2) and the team's own experience show six problems: listings are scattered across informal channels, nobody verifies owners or listings, rooms cannot be inspected remotely, rents are not visible side by side, house rules are agreed verbally, and there is no record of previous tenants' experience. The result is lost time at the start of each semester, wasted trips, overpayment by new students, and disputes after moving in. A wrong choice affects a student's study and safety for a whole year.

2.3 Fundamental concepts behind the problem

User	Need
Current IUT Student	Find a suitable, fairly priced room quickly, and trust the owner and the listing
Newly admitted student	Find a room before classes start, before receiving an IUT e-mail
Departing tenant or student seeking a roommate	Pass the room on, or fill a free seat
Homeowner	Fill empty rooms between semesters with verified students
Administrator	Keep fake owners and listings off the platform

2.4 Specialist knowledge

Knowledge	Why it is necessary here
Bangladeshi land records (Khatian)	Owners are outside IUT, so they need a stronger check than students. The Khatian certificate names the owner of a plot, its number is unique per property, and the admin can verify it, in person if needed.
IUT admission and e-mail practice	Current students have an @iut-dhaka.edu address. Newly admitted students have only a merit rank, so a temporary rank account with a six-month deadline was needed.
Local rental practice	Gate closing time, advance payment, the reason the previous tenant left, and facilities such as IPS (backup power) and gas lines are what students ask about before renting (Section 4.2).
Bangladeshi phone numbers	Owner and student contact numbers are validated against the 11-digit local mobile format.
Geographic coordinates	GeoJSON stores longitude before latitude. Mixing the order up put pins in the wrong place during development (Section 6.6).

Web security	Session cookies, password hashing, input whitelisting and upload limits protect students' and owners' personal data (Section 8).

3. Literature and Existing Solutions

3.1 Existing ways to find a room

Solution	What it Offers	What it lacks for IUT students
Facebook groups (To-Let, batch and university groups)	Free, large audience, quick posts	No owner verification, posts get buried, no structured filters, no map, no reviews
To-Let signs and walking around Board Bazar	Real rooms, direct contact	Takes days, no comparison, no information before the visit
Word of mouth	Local knowledge	Fees, limited choice, information depends on one person
Bangladeshi classified and property sites (for example Bikroy.com's property section, bdHousing.com)	Structured listings with photos and filters across the country	Aimed at families and whole flats, no student or university focus, no ownership check visible to the renter, no tenant reviews
International University off-campus housing portals (for example Places4Students, used by universities in Canada and the USA)	Listings near one campus, restricted to its students	Not available in Bangladesh; owner verification depends on the university
Short-stay platforms (Airbnb)	Verified identities, in-app messaging, two-sided reviews where only guests who stayed can review, and separate ratings for the place and the host	Short stays, online payment and a commercial model that do not fit monthly student rentals in Board Bazar

3.2 Technical sources

The design is based on established industry standards and official technical documentation. Security practices for session management, input validation, and file upload using npm packages and documentation. MongoDB documentation guides the implementation of geospatial indexing using 2d sphere indexes and automatic data expiration using TTL indexes. Mongoose documentation provides the basis for schema inheritance through discriminators, while Socket.IO documentation guides the implementation of real-time communication using rooms and handshake authentication. Additionally, React and Express documentation support the frontend and backend development, respectively. Existing property listing platforms, including Bikroy.com and bdHousing.com , provide references for property listing features, while Airbnb's Help Centre informs the review functionality. Leaflet and OpenStreetMap documentation guide map integration and tile usage, Cloudinary documentation supports image upload and management, and Nodemailer documentation guides email delivery. npm documentation serves as a reference for package management and dependency installation.

3.3 How Khoj builds on and differs from them

- **From Facebook groups and To-Let signs** Khoj keeps the low cost and local focus, and adds verification, structured filters, a map and reviews.
- **From property portals** it keeps structured listings, but restricts the audience to IUT students and adds an ownership check (Khatian) before any listing appears.
- **From student housing portals** it takes the "one campus, verified students only" idea and adapts it to Bangladesh (IUT e-mail, admission rank).
- **From Airbnb** it takes two ideas: reviews only from confirmed stays, and separate ratings for the place and for the host. The difference is that the owner confirms the tenant from the students who messaged about that listing, since there is no booking or online payment.
- **New for this setting:** admission-rank sign-up, Khatian-based owner identity, gate closing time and "reason the previous tenant left" as listing fields, and admin approval of every listing.

The SRS itself was built from user stories and field interviews and described the existing channels only as part of the problem statement. The comparison above was carried out for this report, and it confirms the problem statement: no existing option combines owner verification, a student-only audience and reviews from confirmed tenants.

4. Requirements (SRS Session)

4.1 Stakeholders

Stakeholder	Interest	Involvement
IUT students (current and newly admitted)	Find a verified room quickly at a fair price	Interviewed (3), primary users
Homeowners in Board Bazar	Fill rooms with verified tenants	Interviewed (1), users
Administrator (IUT-side)	Keep the platform free of fake owners and listings	User
Departing tenants	Hand a room over, or find a roommate	Users (student posts)
Supervisor and SPL committee	A working, well-documented system	Reviewed each session

4.2 Elicitation

Requirements came from structured face-to-face interviews held in Board Bazar and on campus, using a 22-question questionnaire in five parts (background, search experience, feature priorities, homeowner questions, general). Three students (2nd, 3rd and 4th year, living in rented flats and the IUT hall) and one homeowner took part, and the answers were collected in a shared form and sheet. Students rated thirteen candidate features on a four-point scale.

Feature	Rated "must have" or "important"
Login restricted to verified IUT e-mail	3 of 3
Map location and distance from the IUT gate	3 of 3
Filter by rent range	3 of 3

Photos organised by room, bathroom, balcony, kitchen	3 of 3
Finding or advertising for a roommate	3 of 3
Availability status	2 of 3
Reviews from previous tenants	2 of 3
Wishlist with vacancy notification	2 of 3
Filters by amenities and by room type	1 of 3 each
In-app chat with the owner	1 of 3
Monthly bill breakdown, reporting fake listings	0 of 3

All three students said they would trust reviews more if only real tenants could write them, and would rather chat in the app than call an unknown number. The homeowner named empty rooms between semesters as their main problem. On a 1 to 5 scale of how likely they were to use Khoj, students answered 4, 4 and 3, and the homeowner 5. The interview records, responses and priority summary are available at https://drive.google.com/drive/folders/1M6hqjxuBFliLY20R3cFhtEdtvDcRFCC8.

4.3 Functional requirements

ID	Requirement	Priority
FR_1	Students register and log in securely with their IUT e-mail, or with their admission rank if newly admitted.	High
FR_2	Homeowners apply with a Khatian certificate; the admin approves or rejects the application.	High
FR_3	Students search and filter listings by rent, facilities, distance and room type.	High
FR_4	Each listing is shown on an interactive map, with its distance from the IUT gate.	Medium

FR_5	Each room shows whether it is available or rented.	Medium
FR_6	Students wishlist rooms and are notified when one becomes available.	Medium
FR_7	Homeowners and students create listings; the property owner manages availability.	High
FR_8	Students and homeowners chat in real time.	High
FR_9	A student whom the owner confirmed as tenant rates and reviews the room and the owner.	High
FR_10	Students report fake listings and the admin acts on reports.	Low
FR_11	A student posts a room they share or are leaving, so another student can take it or join as a roommate.	Low
FR_12	The admin manages users, approves listings and sends notifications.	High

The 28 user stories that refine these requirements, with acceptance criteria and final status, are listed in Section 11.1.

4.4 Non-functional requirements

ID	Category	Requirement
NFR_1	Security	Passwords are never stored in plain text.
NFR_2	Security	Session tokens cannot be read by page scripts.
NFR_3	Security	E-mail ownership is proven before a student account is created.
NFR_4	Security	Uploads are limited to safe file types and sizes.

NFR_5	Security	Client input cannot change database queries or claim another owner's identity.
NFR_6	Privacy	Only the two participants can read a conversation.
NFR_7	Privacy	Owners' Khatian numbers and password hashes are never shown to students.
NFR_8	Performance	Listing, map and inbox pages load without per-item database queries.
NFR_9	Usability	The interface works on phones and desktops, in light and dark themes.
NFR_10	Usability	Validation errors name the field and say what to fix.
NFR_11	Reliability	A failed upload never leaves a half-approved record.
NFR_12	Maintainability	Routing, validation, business rules and data access are separate layers.
NFR_13	Portability	Secrets, addresses and limits are configured without code changes.

How the implementation meets each NFR is shown in Section 11.2.

4.5 Traceability: requirement to stakeholder need

Requirement	Stakeholder need (source)	User stories
FR_1, NFR_1 to NFR_3	Only real IUT students on the platform (students, 3 of 3 rated it important)	US_1, US_2, US_21

FR_2, NFR_7	Trust in the owner (students; one met a fake offer)	US_3, US_4, US_22, US_26
FR_3	Find rooms within budget quickly (rent filter 3 of 3)	US_5, US_6
FR_4	Judge location and commute (map 3 of 3)	US_7, US_24
FR_5	Avoid contacting owners of taken rooms (2 of 3)	US_8, US_13
FR_6	Do not miss a room that frees up (2 of 3)	US_9, US_10
FR_7, NFR_4	Owners fill empty rooms between semesters (homeowner)	US_11, US_12, US_13, US_23
FR_8, NFR_6	Talk without sharing phone numbers (3 of 3 preferred in-app chat)	US_14, US_15
FR_9	Trust reviews only from real tenants (3 of 3)	US_16, US_28
FR_10	Protection from fake listings (proposal)	US_17, US_18
FR_11	Roommate finding (3 of 3)	US_19
FR_12, NFR_5	Platform kept clean and governed (supervisor feedback, admin)	US_20, US_23, US_25, US_27
NFR_8 to NFR_13	Usable on students' phones, maintainable by a student team	All

4.6 Contextual and sustainable-development considerations

Consideration	How Khoj addresses it
Saves Time	Less time spent searching at the start of the semester, and a safer place to live, support study.

Trust and Reliability	Verified owners, visible rents and written house rules make student housing safer and fairer, and reduce overpayment by new students.
Local context	Gate closing time, admission-rank sign-up and the Khatian check fit how renting and admission work around IUT.
Inclusion	Newly admitted students can join before receiving an IUT e-mail. The interface has a dark mode and works on low-cost phones.

4.7 Feedback in the SRS session and changes made

Feedback	Change made
Proposal: make owner registration software-driven instead of moderators visiting owners	Owners apply online with the Khatian certificate and a house photo; the admin approves.
Proposal: keep the feature list in the presentation clean and minimal	Features grouped by user type.
Proposal: the timeline was too high-level	Week-by-week timeline .
SRS session: how were homeowners interviewed, given that many owners in Board Bazar are local residents with limited literacy?	The interviews were held in Bangla, face to face, and each question was explained in plain words. For the software, the SRS records an explicit assumption that homeowners have the basic digital literacy needed to fill in a form and use chat. This assumption is listed as a limitation (Section 11.3), and a Bangla interface and assisted registration are future work.

List of User Stories

5. System Design (Design Session)

5.1 Design goals

Goal	How the design achieves it	Traces to
Trust by design	Owners and listings stay invisible until the admin approves them; the rule is enforced on the server.	FR_2, FR_12
Real-time communication	Chat runs over Socket.IO with one room per user.	FR_8
Filtering in the database	Rent and facilities are resolved by MongoDB queries, not in the browser.	FR_3
Traceable moderation	Each registration and listing keeps its status, rejection reason and review time.	FR_12
Separation of concerns	View, routing, business rules and data can change independently.	NFR_12

5.2 Use cases

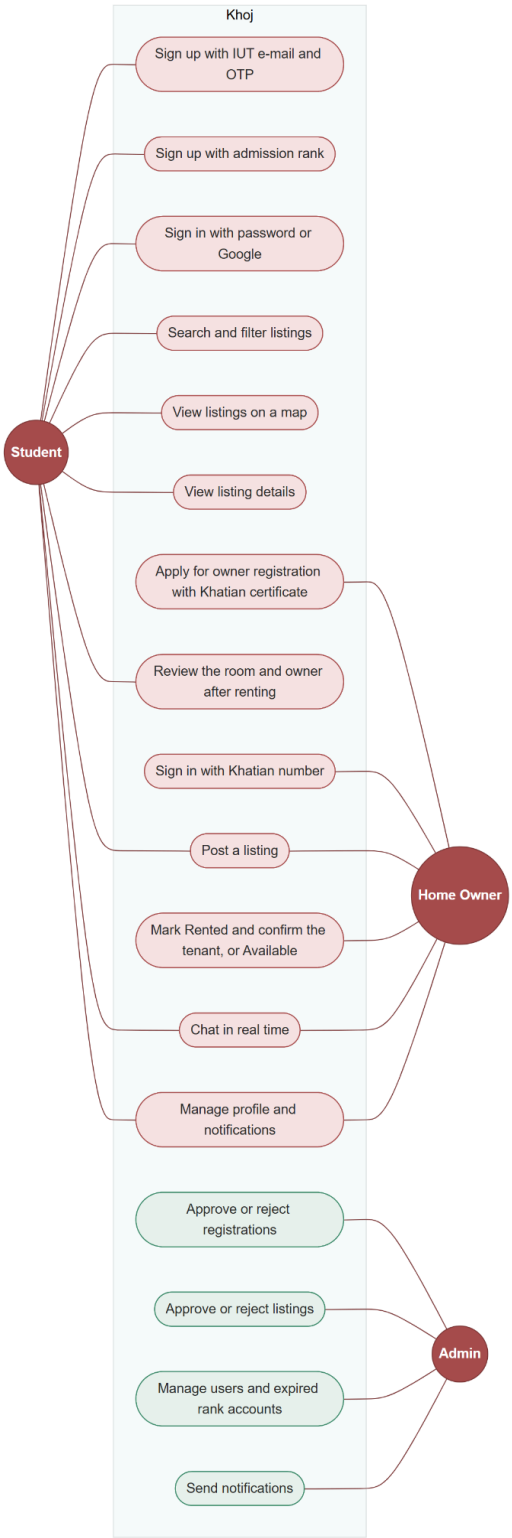


Figure 1: Use case diagram

5.3 Architecture and components

Khoj is a layered client-server system. The React single-page application is the view layer. Express routers, middleware and controllers form the control layer. Services hold the business rules. Mongoose models define schemas, indexes and password hooks. A Socket.IO server is a separate real-time entry point that reuses the chat service. External services (Cloudinary, Gmail SMTP, Google sign-in, OpenStreetMap) are each wrapped in one place.

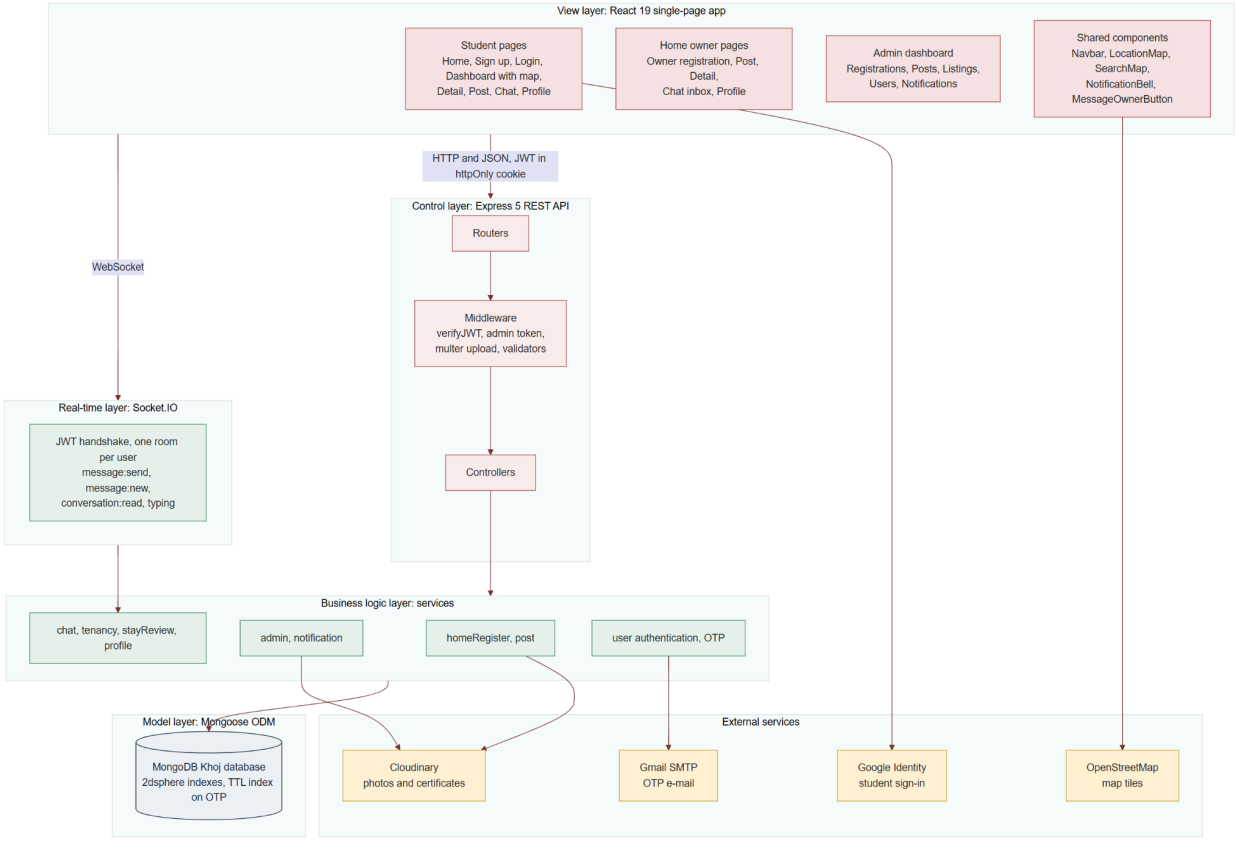


Figure 2: Architecture and component diagram

Component	Responsibility	Traces to

Account pages, user controller, OTP controller	Sign-up, OTP, login, sessions	FR_1
Owner registration page, homeRegister service, admin service	Owner applications and approval	FR_2
Dashboard, filter controller, post service	Search, filters, map	FR_3, FR_4, FR_5
Post page, post service, validation middleware	Creating listings	FR_7, FR_11
Chat page, chat service, chat socket	Messaging	FR_8
Detail page, tenancy and stayReview services	Tenant confirmation and reviews	FR_9
Admin dashboard, admin service, notification controller	Moderation, users, notifications	FR_12

5.4 Data flow

DFD Level: 0

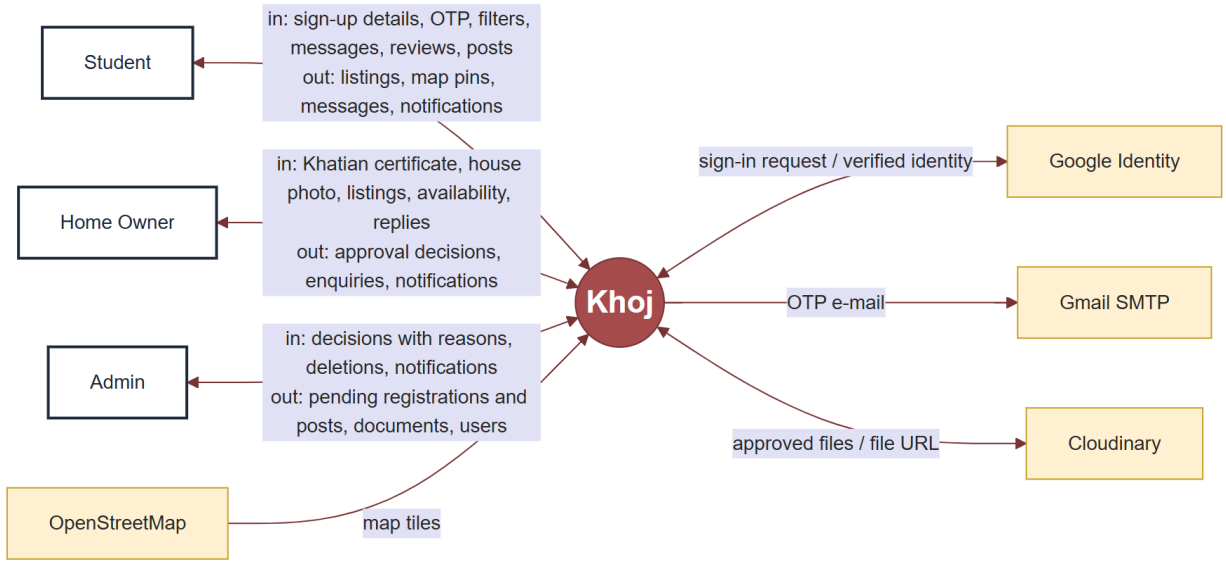


Figure 3: Context diagram (DFD level 0)

DFD Level: 1

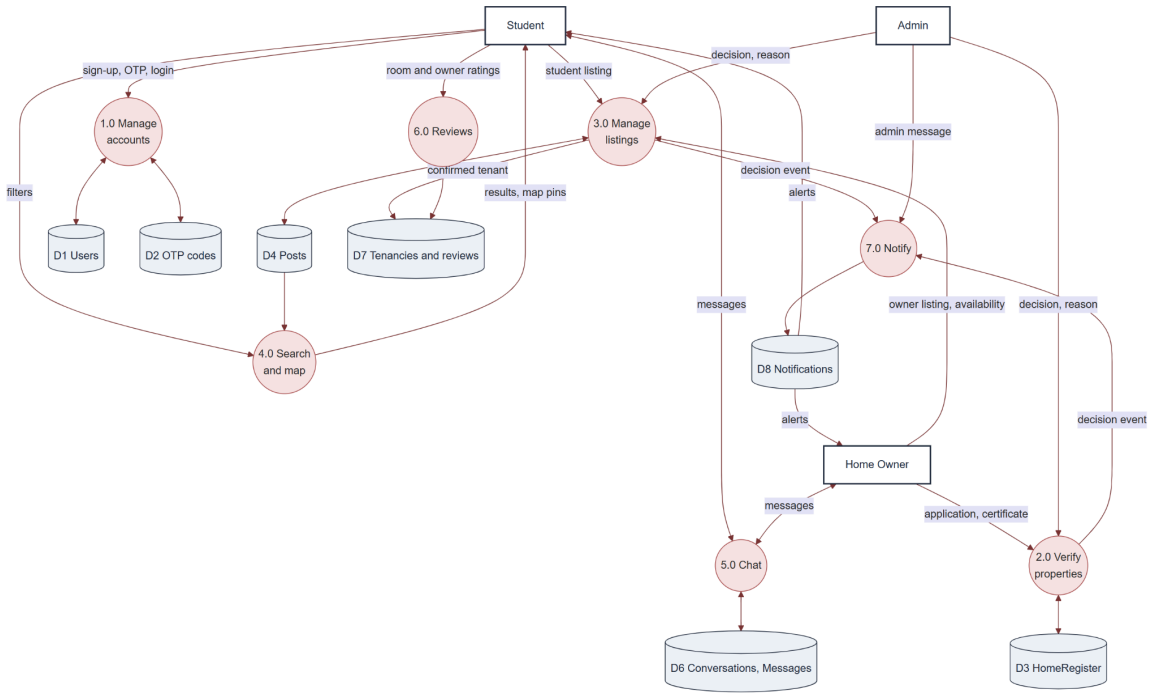


Figure 4: Data flow diagram, level 1

5.5 Module and data model

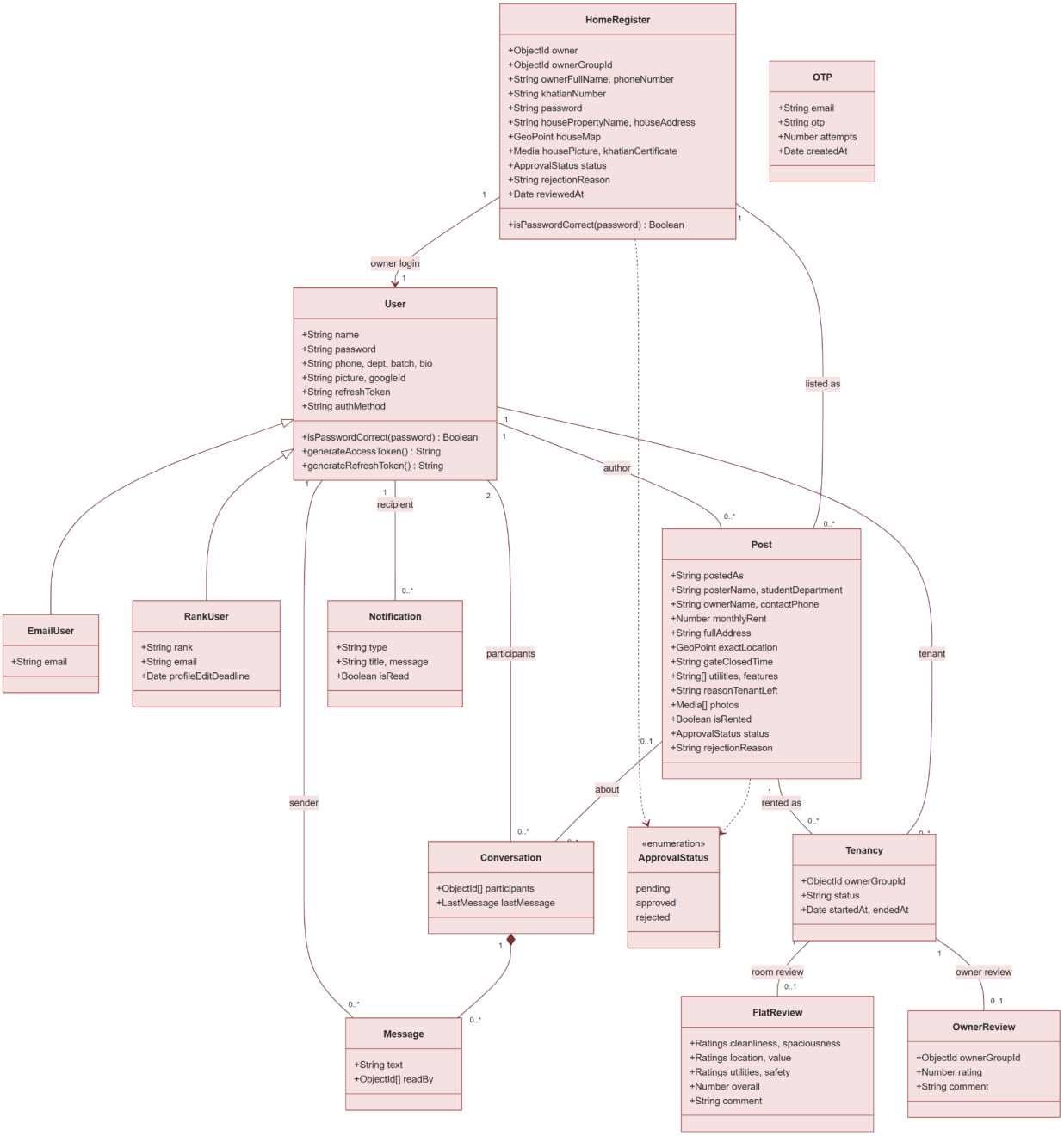


Figure 5: Module diagram data model

Model	Purpose	Traces to
User	Base account: name, hashed password, profile, Google id, refresh token. Issues access and refresh tokens.	FR_1
EmailUser, RankUser	The two student account types, stored in the User collection and told apart by authMethod. RankUser holds the admission rank and the six-month deadline.	FR_1
HomeRegister	An owner application and, once approved, the owner's identity: Khatian number, hashed password, contact e-mail, property, map location, house photo, certificate, moderation fields, and ownerGroupId linking an owner's properties.	FR_2
Post	A listing tied to an approved HomeRegister: poster type, rent, address, location, gate closing time, utilities, amenities, reason the previous tenant left, two photos, rental and moderation status.	FR_3 to FR_7, FR_11
Tenancy	A stay confirmed by the owner. At most one active tenancy per listing.	FR_9
FlatReview	Six 1 to 5 room scores, their average and a comment. One per tenancy.	FR_9
OwnerReview	A 1 to 5 owner rating and a comment, grouped by owner. One per tenancy.	FR_9
Conversation, Message	A one-to-one thread, optionally about a listing, with a copy of the last message; messages list who has read them.	FR_8
Notification	An in-app alert for one user.	FR_12
OTP	A one-time code for an e-mail address, deleted automatically after five minutes.	FR_1

5.6 Database schema

MongoDB stores ten collections. References are ObjectIds; the arrow points from the document that holds the reference.

Collection	Key fields	References	Indexes
users	name, password (bcrypt), authMethod, email or rank, profileEditDeadline	none	unique email, unique rank
homeregisters	khatianNumber, password (bcrypt), email, houseMap (Point), status	owner → users	unique khatianNumber, 2dsphere on houseMap
posts	postedAs, monthlyRent, exactLocation (Point), utilities[], features[], photos[], isRented, status	author → users, registeredHouse → homeregisters	2dsphere on exactLocation, status, isRented
tenancies	status (active, ended), startedAt, endedAt, ownerGroupId	post → posts, student → users	unique active tenancy per post (partial index)
flatreviews	ratings (6), overall, comment	tenancy (unique), post	unique tenancy
ownerreviews	rating, comment, ownerGroupId	tenancy (unique)	unique tenancy
conversations	participants[], lastMessage	participants → users, post → posts	participants + updatedAt
messages	text, readBy[]	conversation, sender	conversation + createdAt
notifications	type, title, message, isRead	user → users	user
otps	email, otp, attempts, createdAt	none	TTL 5 minutes on createdAt

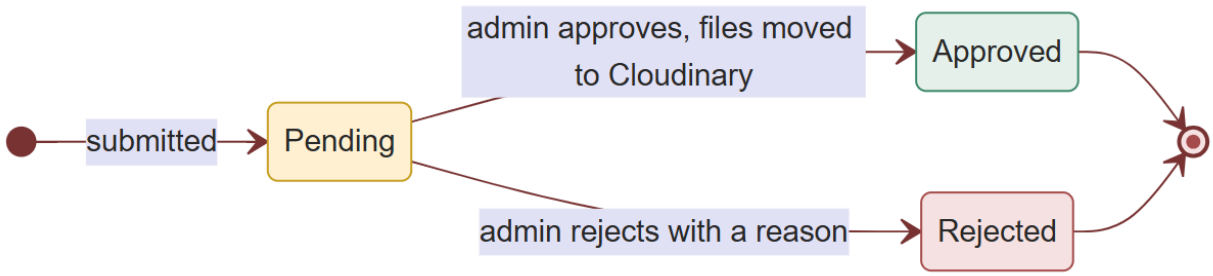


Figure 6: Approval state of a registration or listing

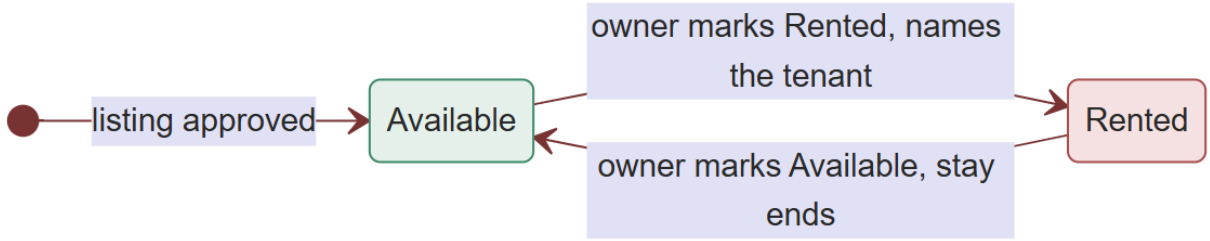


Figure 7: Rental status of an approved listing

5.7 Key interactions

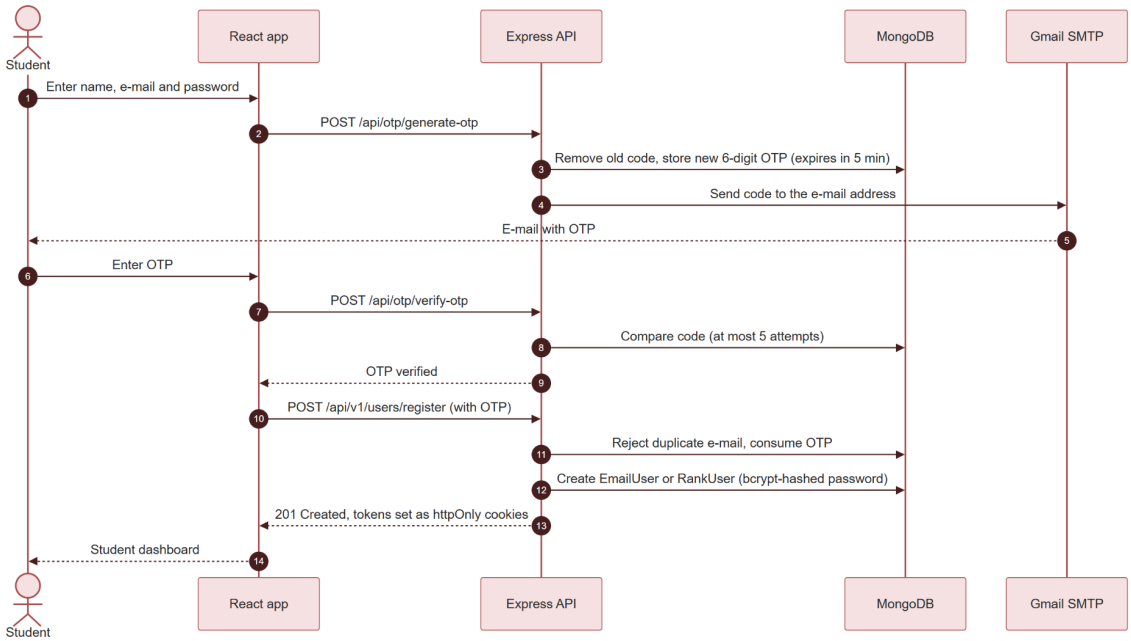


Figure 8: Student sign-up with OTP

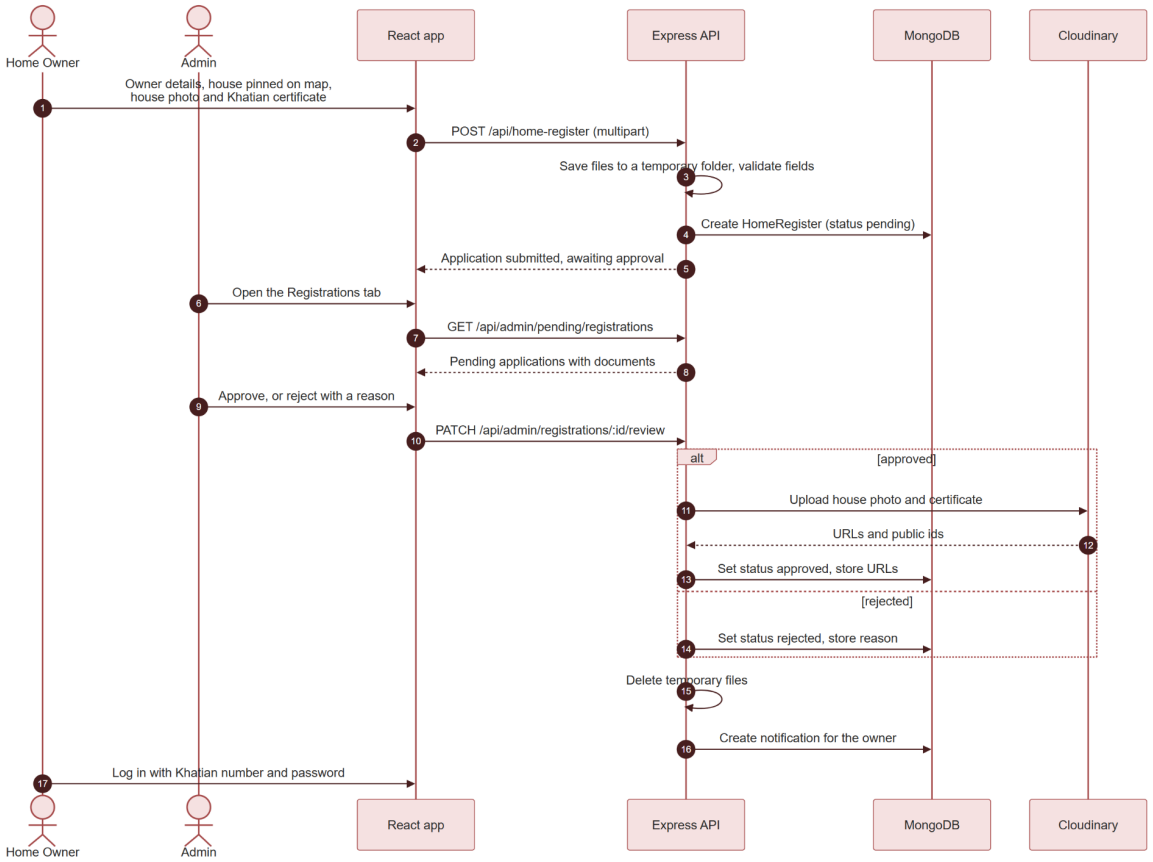


Figure 9: Owner registration and admin approval

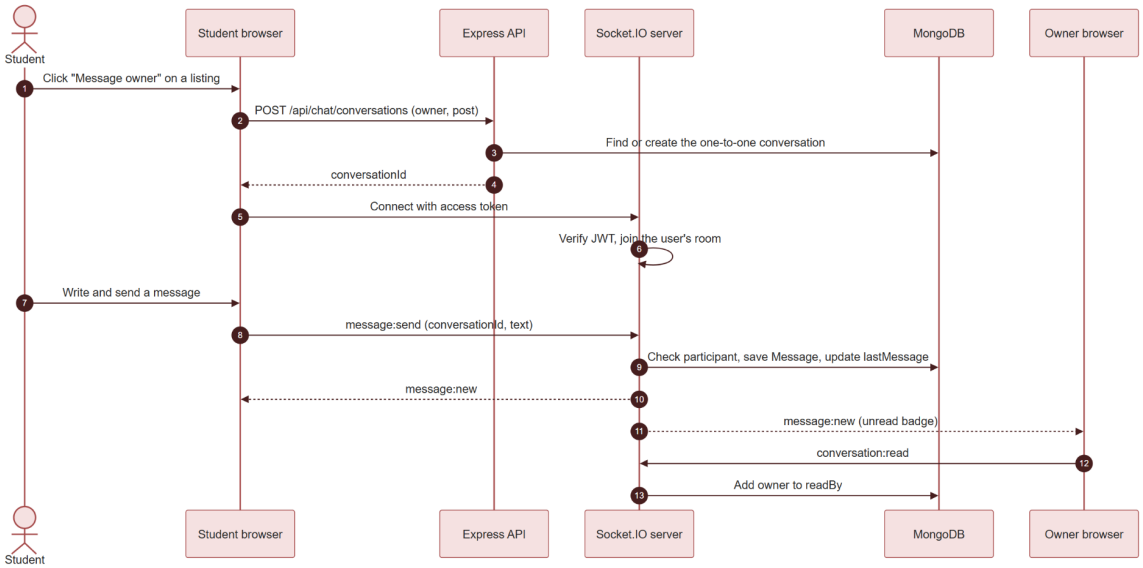


Figure 10: Sending a chat message

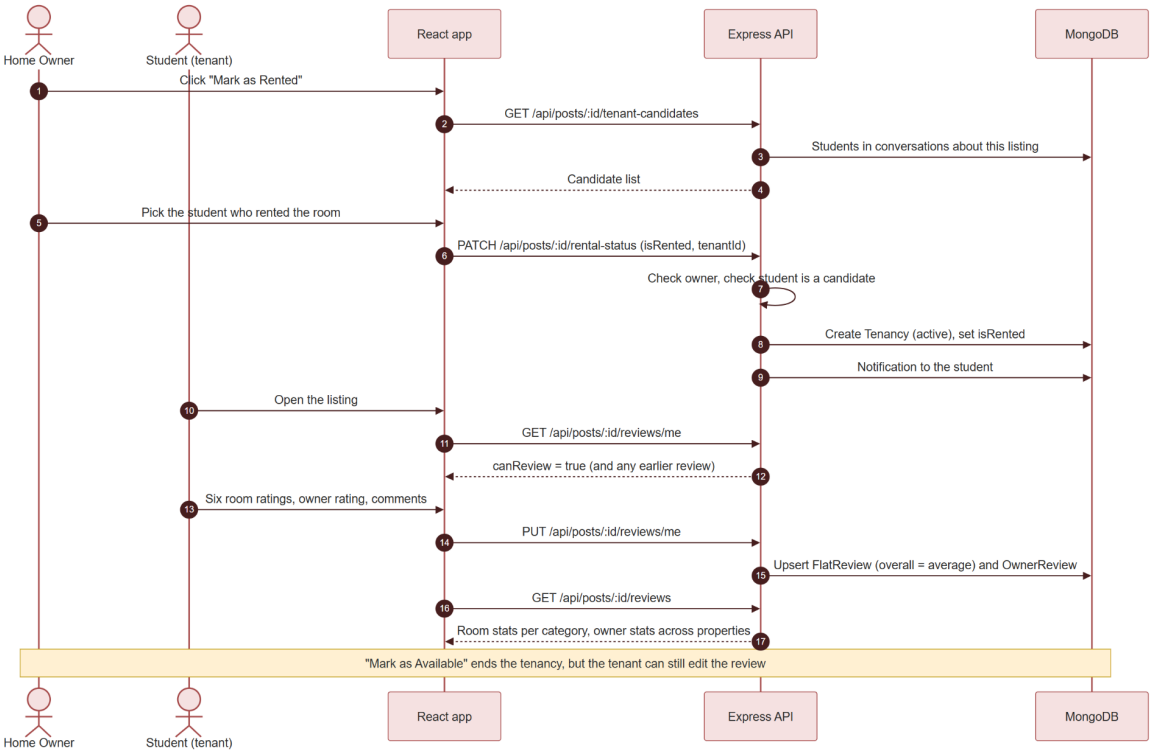


Figure 11: Confirming a tenant and reviewing the stay

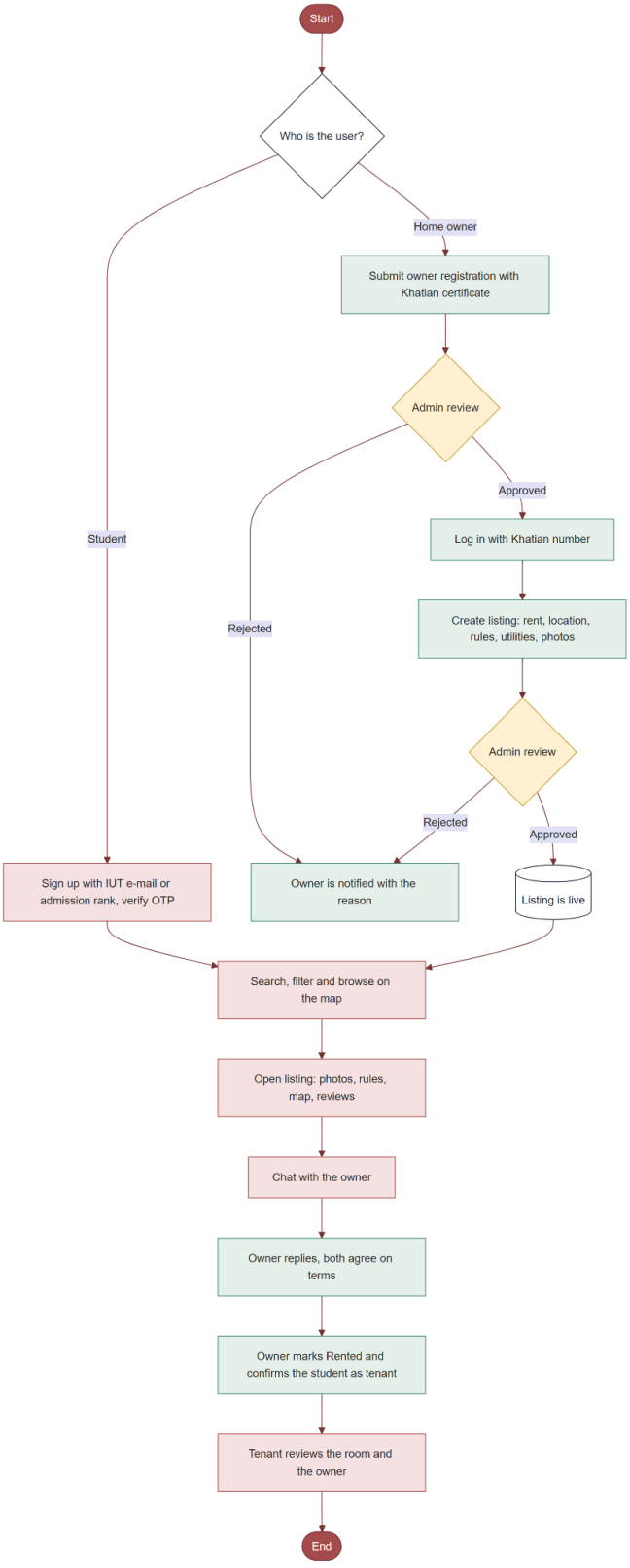


Figure 12: End-to-end workflow

5.8 Design alternatives

Each decision below compares the chosen design with at least one plausible alternative, against stakeholder needs, usability, scalability, resource use and lifecycle cost.

A. Overall architecture: layered monolith vs. microservices

Criterion	Layered monolith (chosen)	Microservices (accounts, listings, chat)
Stakeholder needs	All features in one deployable app, enough for one campus	Same features, more moving parts
Usability	No difference to users	No difference
Scalability	Scales vertically; Socket.IO would need a shared adapter to scale out	Each service scales on its own
Resource use	One Node process, one database	Several processes, a gateway, inter-service calls
Lifecycle cost	One codebase a three-person team can maintain	Higher deployment and debugging cost

A monolith with clear layers fits the size of the user base (one campus) and the team. The layers keep a later split possible.

B. Database: MongoDB vs. PostgreSQL

This comparison was raised by the industry expert in the Design session

Criterion	MongoDB with Mongoose (chosen)	PostgreSQL (with PostGIS)
Stakeholder needs	Listings have optional and list-valued fields (utilities, amenities, photos) that grew as students asked for more	Array or JSON columns, or join tables, for the same fields
Data integrity	No foreign keys; every reference (listing → property, review →	Foreign keys, unique and check constraints enforced by the database itself

	tenancy) is checked in application code	
Consistency	Single-document writes are atomic; multi-document transactions need a replica set and are not used	ACID transactions across tables by default, for example approving a listing and creating its notification together
Querying	Lookups between collections through populate and aggregation pipelines	SQL joins, mature query planner and reporting tools
Scalability	Built-in geospatial and TTL indexes; horizontal sharding	PostGIS geospatial support; vertical scaling and read replicas are enough for one campus
Resource use	Free local server or free cloud tier	Free local server and free managed tiers
Lifecycle cost	One language (JavaScript) end to end; the team already knew the MERN stack	A second language (SQL) and migrations for schema changes, but the schema documents itself

Why PostgreSQL is the industry standard for this kind of data?

Khoj's data is strongly relational: owners, properties, listings, tenancies, reviews and conversations all point to one another, and most rules are about those links (a review needs a tenancy, a listing needs an approved property, one active tenant per listing). A relational database enforces such rules in the schema, keeps related writes consistent with transactions, and lets anyone with SQL query the data. It has decades of tooling for backups, migrations and reporting, and PostGIS covers the map features . Document databases fit best when records are self-contained and their shape changes often, which describes Khoj's listing details but not the rest of its data.

Decision:

MongoDB had been chosen at the start for its flexible documents and the team wanted to become familiar with the MERN stack as there was already a course on postgreSQL. When the alternative was raised, most of the backend was already built on Mongoose. After discussing it with the supervisor, the team kept MongoDB: rewriting the data layer at that stage would have put the remaining features at risk. The team acknowledges that a relational database should have

been considered at the design stage. The cost of the choice is weaker integrity, which the code makes up for with server-side checks and unique and partial indexes (for example one active tenancy per listing, one review per tenancy).

C. Media storage: upload after approval vs. upload on submission

Criterion	After approval (chosen)	On submission
Stakeholder needs	Admin still sees the files before deciding, through an admin-only file route	Admin sees cloud URLs immediately
Resource use	Rejected content never uses cloud storage	Every rejected upload uses free-tier quota
Reliability	Status changes only after every upload succeeds	Orphaned files when an application is rejected
Lifecycle cost	Temporary files must be cleaned up after the decision	Cleanup of rejected files in the cloud

D. Real-time chat: Socket.IO vs. HTTP polling

Criterion	Socket.IO (chosen)	Polling every few seconds
Usability	Instant delivery and live unread counts	Delay up to the polling interval
Resource use	One open connection per tab	Many empty requests
Lifecycle cost	Separate authentication at the handshake	Simpler server

E. Owner identity: Khatian number vs. e-mail account

Criterion	Khatian number and certificate (chosen)	E-mail and password
Stakeholder needs	Proves land ownership, which students care about most	Proves only an e-mail address
Usability	Owners know their Khatian number; one login per property	Familiar, one login per person
Lifecycle cost	Needs the owner-group link for owners with several houses	Simpler

5.9 Feedback in the Design session and changes made

Feedback	Response
The industry expert suggested a relational database such as PostgreSQL instead of MongoDB, as the industry-standard choice	Discussed with the supervisor. MongoDB was kept because the data layer was already built; the trade-off is analysed in Section 5.8 B, and integrity rules were enforced in code and indexes.
Hand-drawn diagram fonts were hard to read	Clean, coloured diagrams in later documents and the final presentation

Design-phase models that were changed in implementation are listed in Section 6.7.

6. Implementation (Implementation Session)

6.1 (a) Project structure

The repository holds two applications: backend/ (Node.js and Express API) and my-app/ (React client). Each has its own package.json, so the two can be installed, run and deployed separately. The tree below is the output of the tree with node_modules, build, public and stylesheet files excluded. The full tree is in khoj_file_tree.txt.

```
SPL-2-23/
├── backend/
│   ├── .env.sample
│   ├── package.json
│   └── src/
│       ├── controllers/  11: request in, service call, reply
│       ├── db/           database connection
│       ├── middleware/   7: JWT, admin token, uploads, checks
│       ├── models/       12: Mongoose schemas
│       ├── routes/       11: URL to middleware and controller
│       ├── scripts/      3 one-off data migrations
│       ├── services/     7: business rules
│       ├── socket/       Socket.IO chat server
│       ├── utils/        7: shared helpers
│       ├── app.js        Express app and route mounting
│       ├── constants.js  database name, IUT gate location
│       └── index.js      starts HTTP and Socket.IO servers
└── my-app/
    ├── .env.sample
    ├── package.json
    └── src/
        ├── pages/        10 screens
        ├── components/   8 shared interface pieces
        ├── App.js        switches between pages
        ├── config.js     server address (API_BASE)
        ├── auth.js       session storage
        ├── chatApi.js    chat REST calls
        ├── socket.js     one shared Socket.IO client
        └── chatNotifications.js  unread chat tracking
```

The ten pages are:

Home, Signup, Login, HomeRegister, Dashboard, Detail, Post, Chat, Profile and AdminDashboard. The eight components are Navbar, LocationMap, SearchMap, NotificationBell, ChatNavButton, MessageOwnerButton, StayReviews and TenantPicker.

The utilities are:

ApiError, ApiResponse, asyncHandler, cloudinary, cors, distance and removeLocalFile.

Why this structure?

The backend is split by layer, and each layer has one job . A feature is a vertical slice through the layers, with matching names (post.route.js, post.controller.js, post.service.js, post.model.js), so a developer can find any feature's code by its name. The frontend is split into pages (one per screen) and components (reused, or pulled out of a page to keep it readable), and only a few transport files talk to the server.

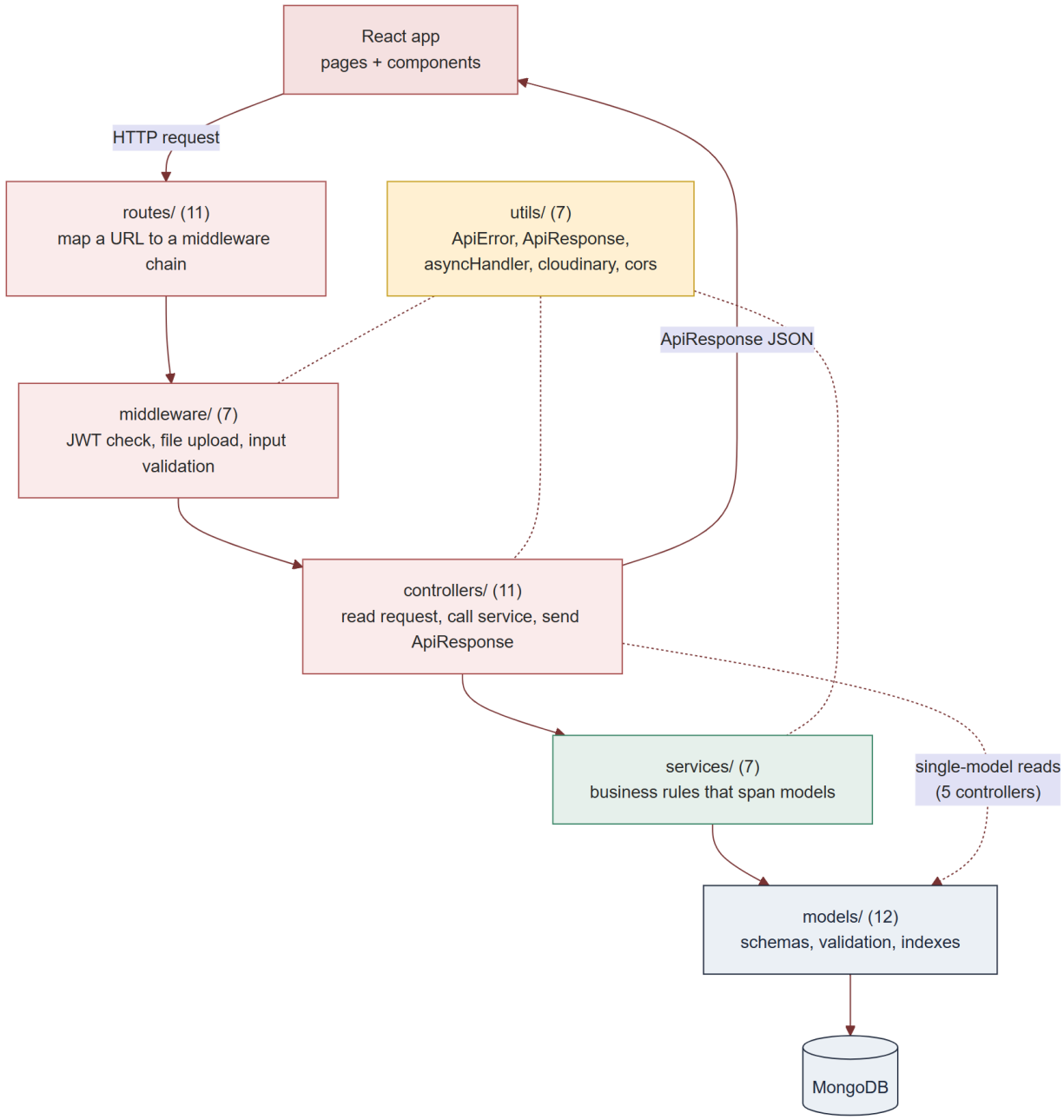


Figure 13: Backend layers and their jobs

6.2 (b) Communication between components

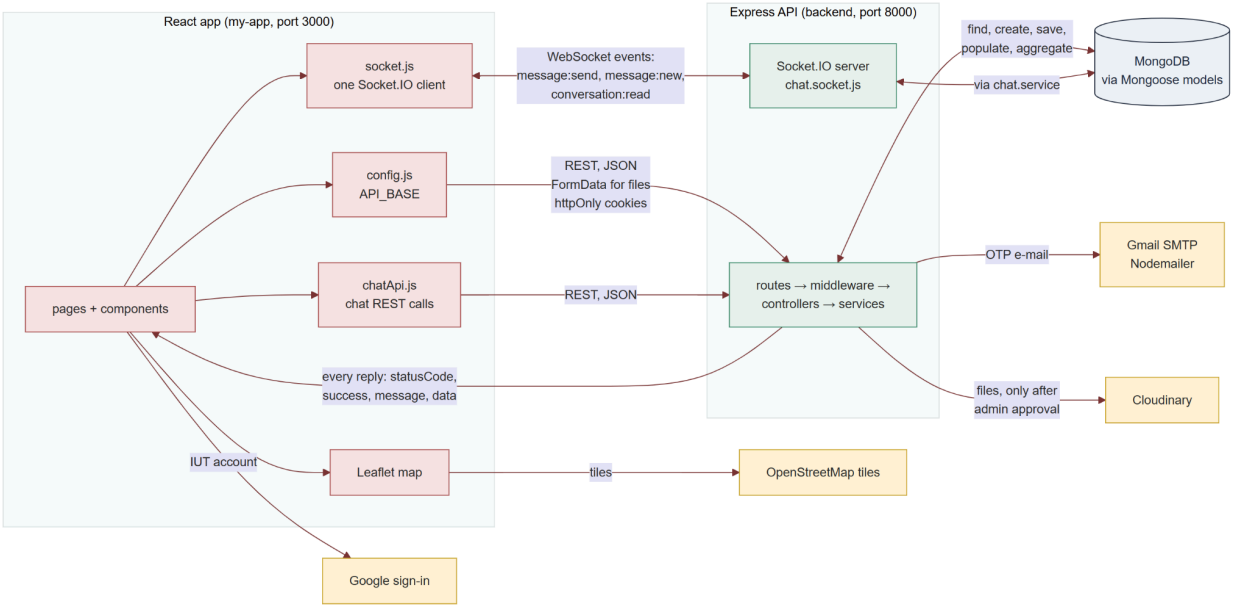


Figure 14: How the components talk to each other

From → to	Protocol	Format
React → Express	HTTP REST	JSON bodies; multipart/form-data for files; session in httpOnly cookies (`accessToken`, `refreshToken`)
Express → React	HTTP REST	Every success is an ApiResponse: `{ statusCode, data, message, success }`. Every error is an ApiError turned into `{ statusCode, message, success: false, errors }` by one error handler.
React ↔ Socket.IO server	WebSocket	Events: client sends `message:send` and `conversation:read`; server sends `message:new`. The server also handles a typing

		event that the client does not use yet. The JWT is checked at the handshake.
Express → MongoDB	Mongoose	Model documents (Section 5.6): find, create, save, populate, aggregate
Express → Cloudinary	HTTPS SDK	File path in; secure_url and public_id out, stored as `{ url, publicId }`
Express → Gmail SMTP	Nodemailer	OTP and rejection e-mails
React → Google, OpenStreetMap	HTTPS	Google sign-in credential; map tiles

Data classes exchanged. JavaScript has no separate DTO classes here. The request shape is fixed by the validation middleware, which turns the raw form into `req.validatedData` (types converted, lists parsed, unknown values rejected). The response shape is fixed by the ApiResponse class. The database shape is fixed by the Mongoose schemas.

End-to-end example: posting a listing. An owner submits a listing, the admin approves it, and a student sees it . Figure 16 shows the data at each step.

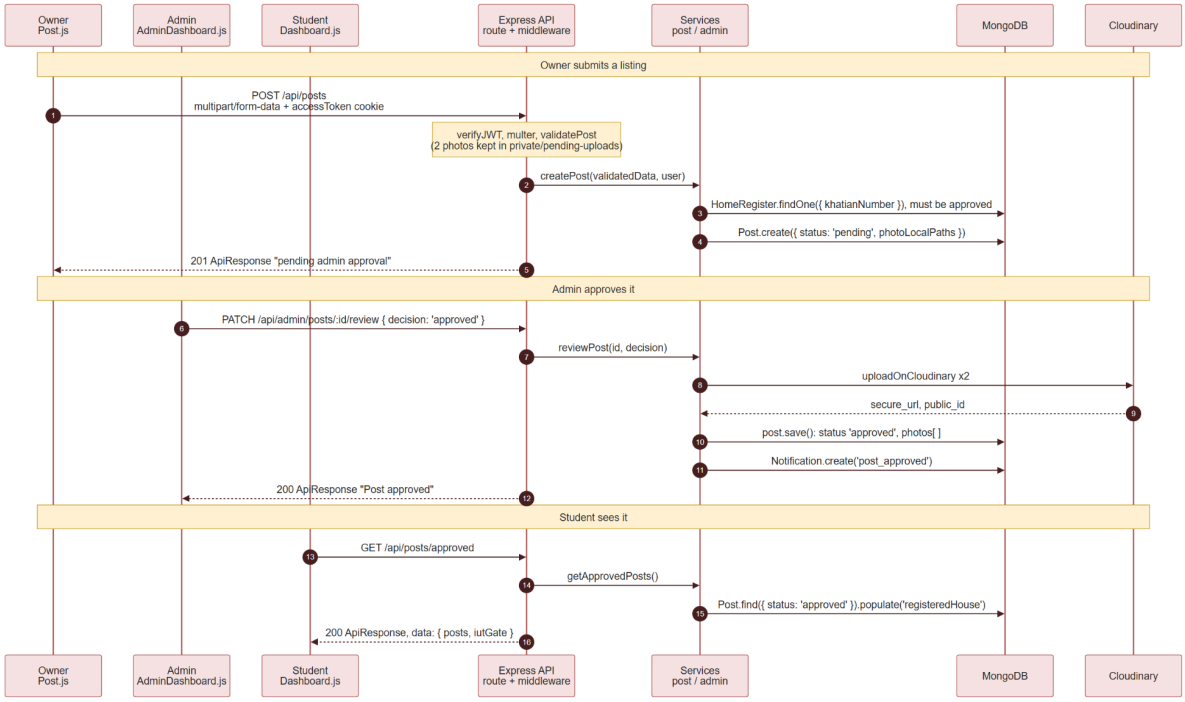


Figure 15: End-to-end request: an owner posts, the admin approves, a student sees the listing

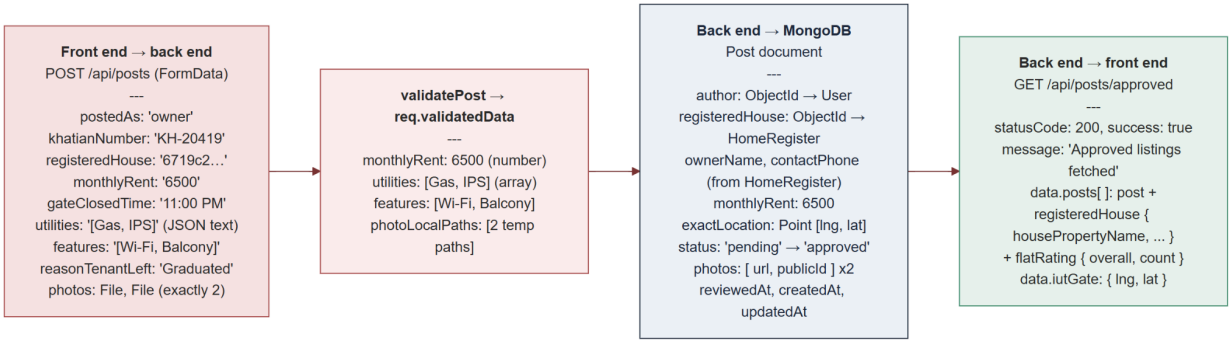


Figure 16: Data exchanged at each step of the example

1. `Post.js` sends `POST /api/posts` as form data with two photos. The browser attaches the access-token cookie.

2. `verifyJWT` identifies the user, multer keeps the photos in private/pending-uploads, and `validatePost` converts and checks every field.

3. `createPost` finds the approved property by Khatian number and copies the owner's name and phone from it, so a listing cannot claim a different owner. It saves the Post as pending.

4. The admin approves through `PATCH /api/admin/posts/:id/review`. `reviewPost` uploads both photos to Cloudinary, saves the URLs, sets the status to `approved` and creates a notification for the author.

5. The student's dashboard calls `GET /api/posts/approved` and receives the listing with its property name and room rating.

6.3 (c) Division of modules and methods

The code is JavaScript: Express on the server and React on the client. It has two real classes, `ApiError` (extends Error) and `ApiResponse`. Everything else is organised as modules (one .js file each) and functions. We therefore report modules in place of classes, and count as a method every named function: function declarations, functions stored in a constant, handlers wrapped in `asyncHandler`, schema methods, React components and the functions inside them.

Rationale:

- **Layering and single responsibility.** `Routes` only map URLs (16.8 lines on average). `Middleware` only checks: login, files, input. `Controllers` only translate between HTTP and services (methods average 11.5 lines). `Services` hold rules that span several models (approval, tenancy, reviews, chat). `Models` only describe data (methods average 3.8 lines).
- **When a service exists.** A service is added when a feature has rules across several models. Five controllers that do single-model reads (OTP, notifications, owner info, filters, login) talk to models directly.
- **Reuse.** `asyncHandler` removes try/catch from 41 handlers; `ApiError` and `ApiResponse` give one error and reply shape across the backend; `verifyJWT` protects 7 route files; `createMessage` in the chat service is used by both the REST route and the socket event; Navbar appears on all 10 pages; `config.js` holds the server address for 8 frontend files.
- **Inheritance where it helps.** `EmailUser` and `RankUser` extend `User` as Mongoose discriminators, so password checking and token creation are written once.

Lines of code

Counted with a Babel-parser script on the final main branch: non-blank, non-comment lines in backend/src and my-app/src, excluding dependencies, build output, stylesheets and the three one-off migration scripts.

Modules	Count	Average LOC	Maximum LOC	Largest
Backend	60	49.3	220	services/admin.service.js
Frontend	26	232.8	755	pages/HomeRegister.js
Whole project	**86**	**104.8**	**755**	pages/HomeRegister.js

Methods	Count	Average LOC	Maximum LOC	Largest
Backend	146	13.3	77	validatePost
Frontend, all	226	30.5	621	HomeRegister (component)
of which React components	53	101.0	621	HomeRegister
of which functions inside components	173	8.9	109	submit in Login.js
Whole project	**372**	**23.7**	**621**	HomeRegister

The median method is 8 lines and the median module 49 lines. In the backend, 84% of methods are 25 lines or shorter and none exceeds 100.

Justification of the maximums

A React component is one function that returns its whole screen, so a full page counts as one method. About two thirds of the largest components are markup: 472 of HomeRegister's 621 lines, 380 of Post's 587. Their logic is already split into small inner functions, which average 8.9 lines. The largest backend method, validatePost (77 lines), checks every field of the listing form in one place so that all errors are returned together. The largest backend module, admin.service.js

(220 lines), holds all admin actions behind one service. Splitting the long form pages into section components is planned .

6.4 Implemented features

Area	Feature
Accounts	IUT e-mail sign-up with e-mailed OTP; admission-rank sign-up with personal e-mail OTP and six-month deadline; Google sign-in for IUT accounts; login with e-mail, rank or Khatian number; httpOnly cookie sessions with refresh
Owners	Registration with Khatian certificate, house photo, map pin, phone and e-mail; admin preview of the files before deciding; approval moves files to Cloudinary; rejection reason e-mailed to the owner; linking another property to the same owner
Listings	Four-step posting form (photos, details, facilities, preview). Posting as an owner (Khatian re-check) or as a student for an approved property (roommate post, with department and phone); owner name, address and map location always copied from the verified property; two photos (room and washroom); rent, gate closing time, utilities, amenities, reason the previous tenant left; admin approval; owner marks Available or Rented
Search	Filter bar with post type (roommate posts by students, house-rent posts by owners), status (rented listings hidden by default, except the viewer's own), rent presets or a custom range, and multi-select utilities and amenities, with removable filter chips; rent and facilities resolved in one database query; cards and price-labelled map pins side by side, listings at one spot grouped into one pin; IUT gate marker; cards show who posted (owner, or student with department and phone)
Chat	Conversation per student, owner and listing; message the owner, or the student who posted a roommate listing; real-time delivery to every open tab; history; unread conversations marked in the inbox and counted on the chat button; works across devices on the same network
Trust	Owner names the tenant from students who messaged about the listing; tenant reviews the room on six categories and the owner on one scale; editable;

	category averages on the listing; owner rating combined across linked properties
Admin	Dashboard tabs for pending posts and registrations, live listings, users and notifications; approve or reject with reason; delete users; list expired rank accounts; send notifications
Profile and UI	Editable student profile and picture; owner picture; notification bell; responsive layout; dark mode

6.5 Fundamental concepts in the code

Concept (Section 2.3)	Where it appears
Inheritance	`User` base schema with `EmailUser` and `RankUser` discriminators; `ApiError` extends `Error`
Hashing	`bcrypt pre("save")` hooks in `user.model.js` and `homeRegister.model.js`; `isPasswordCorrect`
Tokens	`generateAccessToken` and `generateRefreshToken` (JWT); `verifyJWT` middleware; socket handshake check
State machines	`status` enum `pending, approved, rejected` on `Post` and `HomeRegister`; Tenancy `active` and `ended`; `isRented`
Sets and arrays	Filters use `$all` so a listing must hold every selected facility; `readBy` arrays give unread counts without a counter
Indexes	`2dsphere` on locations; TTL index deletes OTPs after five minutes; partial unique index allows one active tenancy per listing; compound indexes for the inbox and message history
Aggregation	`stayReview.service.js` groups reviews with `$group` and `$avg` per category and per owner group

Denormalisation	`lastMessage` stored on `Conversation` so the inbox needs no query per thread
Publish and subscribe	Each socket joins a room `user:<id>`; messages are emitted to both participants' rooms
Chain of responsibility	Express middleware chain: verifyJWT → multer → validator → controller
Higher-order functions	`asyncHandler` wraps each handler and forwards errors to one handler
Whitelisting	Filter values are checked against fixed lists and converted to strings before reaching the query

6.6 Challenges and how they were solved

Challenge	Cause	Solution
Map pins in the wrong place	`GeoJSON` stores longitude first; some records had the order reversed	Coordinate handling normalised; a migration script fixed old posts
Other devices could not use the app or chat	Allowed origin hard-coded to localhost	One CORS policy, shared by the REST API and the socket server, allows private network addresses
Owner never learned why a registration was rejected	The owner has no account until approval	Owners give a contact e-mail; the rejection reason is e-mailed
Linking several houses to one owner	Each property has its own login	An owner group id, a migration script, and combined owner ratings
Working in parallel	Merge conflicts and committed dependencies	Personal branches, a shared `.gitignore`, `.env.sample` templates

6.7 Following and departing from the design

Design	Implementation	Reason
Layered architecture, services for business rules	Followed; five single-model controllers skip the service layer	A service adds nothing for one-model reads
Student, HomeOwner, Admin subclasses of User	Student types are discriminators; owners are a `HomeRegister` plus a `User`; the admin uses an `admin key`	Owners log in with a Khatian number, not an e-mail; one admin was enough for one campus
Moderator role	Merged into Admin	Too few users to need two roles
Review with six categories and an owner reply (OwnerResponse)	Two tiers: six-category room review plus an owner rating; no owner reply	Separates the room from the landlord; replies left for future work
RoommateAd model	Student posts ("posting as a student")	Covers both a departing tenant and a roommate search with one approval flow
AvailabilityStatus with "soon vacant"	Available and Rented only	Partly implemented (Section 11)
MediaAsset with photo categories	Two photos, labelled room and washroom	Partly implemented (Section 11)
GeoPoint with stored distance from the IUT gate	Locations and map pins kept; stored distance and travel time removed	The values were straight-line distances, which looked shorter than the real walk by road, and the team was not sure the stored gate point was exact. Misleading numbers were worse than none (Section 9.3).

AdminActionLog	Each record keeps its own status, reason and review time	Enough for one admin
OtpToken storing a hash	OTP stored as given, deleted after five minutes	Short lifetime kept; hashing planned

6.8 Refactoring after the Implementation session

Change	Why	Commit
Single review replaced by tenancy-based two-tier reviews; the old open review endpoint removed	Only confirmed tenants may review; the old endpoint accepted reviews without a login	*35bac9a*
Rent, utility and amenity filters merged into one validated query	One code path; values checked against fixed lists	*35bac9a*
Admin-only routes to view pending files	The admin could not see documents before deciding	*604615b*
Owner contact e-mail and rejection e-mail	Owners did not learn the rejection reason	*604615b*, *3f94a9f*, *99efdd2*
Stored distance model and service removed	Straight-line values did not match real walking distance (Section 6.7)	*5876ca4*, *a1b5080*
Google sign-up checks the account's domain and verified flag in the browser	Only IUT Google accounts should pass; the server-side check is still open (Section 8.3)	*80092ad*

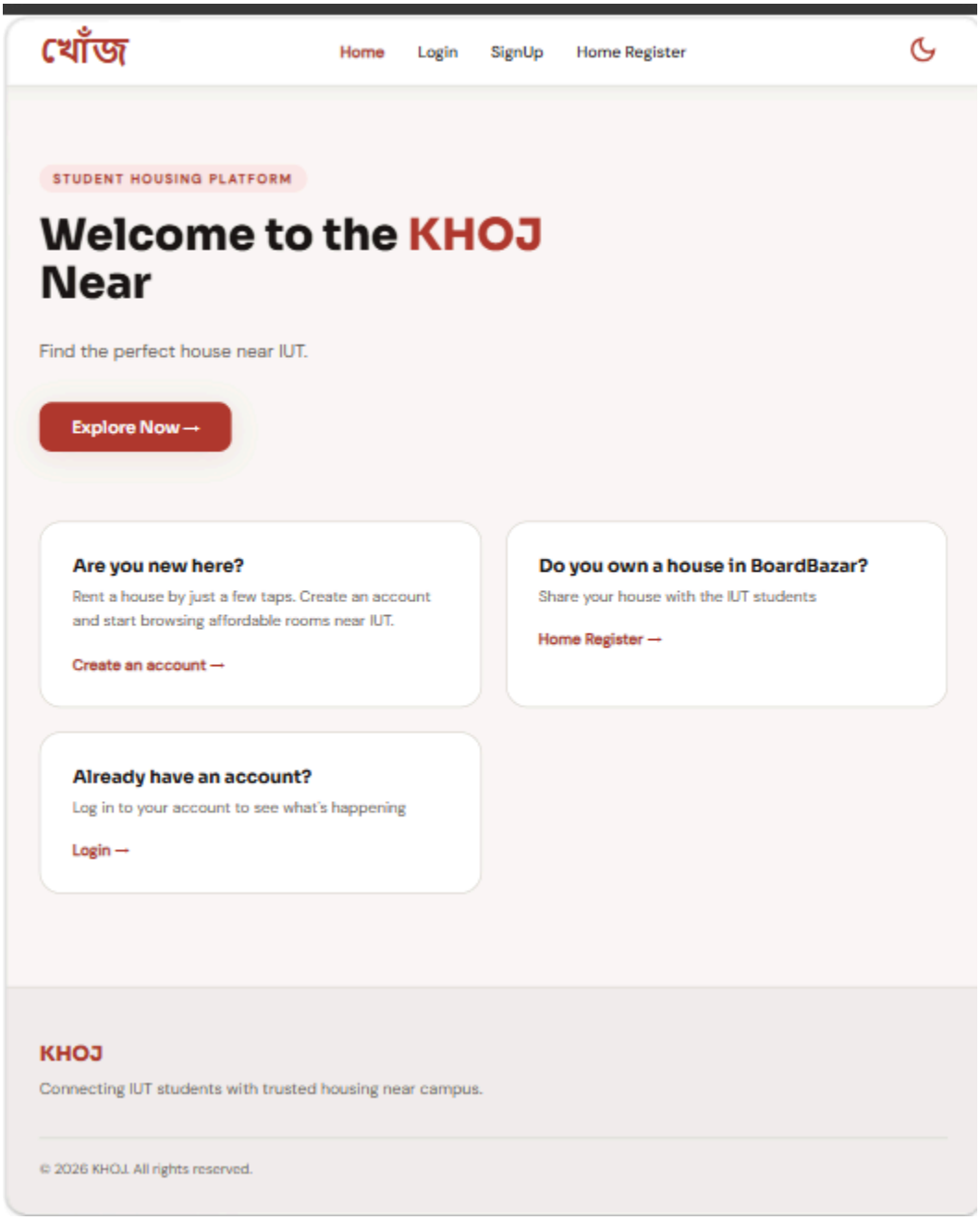
6.9 User interface

The interface uses the brick-red theme from the project's presentations, with a light and a dark mode, and adapts to phone screens.

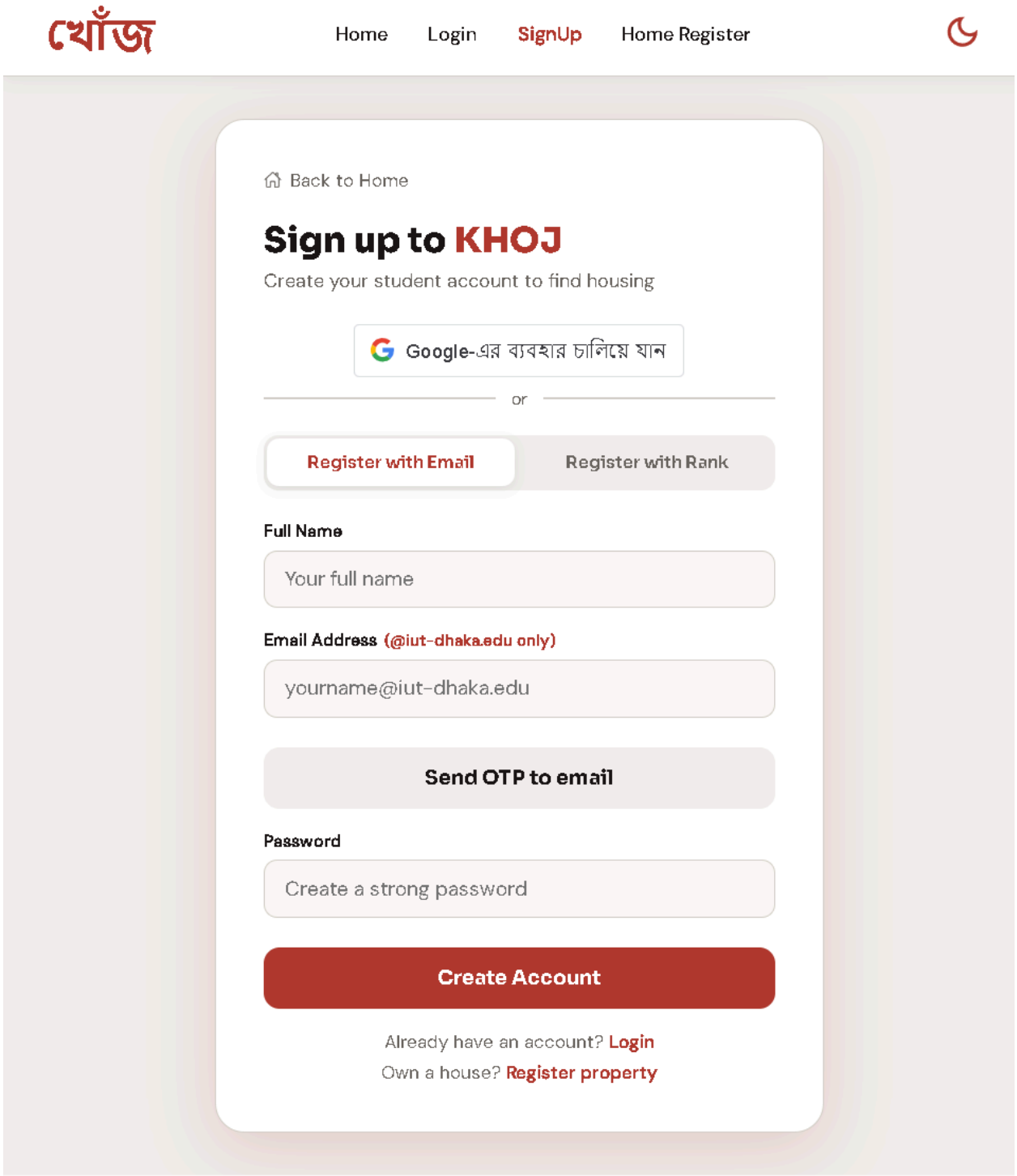
Screen	What the screenshot should show

Landing page	Entry point with sign-up and login for each user type
Student sign-up	IUT e-mail form with the OTP step; the rank sign-up tab
Owner registration	Khatian number, certificate and house photo upload, map pin
Dashboard	Filter bar (post type, status, price, utilities, amenities) with chips, listing cards, map with price pins and the IUT gate marker
Listing detail	Photos, rent, gate closing time, facilities, map, room and owner reviews
Create listing	The four-step form: "I'm the property owner" with the Khatian check, or "I'm a student"
Mark as Rented	Tenant picker listing the students who messaged about the listing
Review form	Six star rows for the room, one for the owner
Chat	Inbox with unread conversations marked, and an open conversation
Admin dashboard	Pending registration with "view certificate", approve and reject with reason
Notifications and profile	Bell with unread alerts; profile editing
Dark mode and phone view	Dashboard in dark mode; a page at phone width

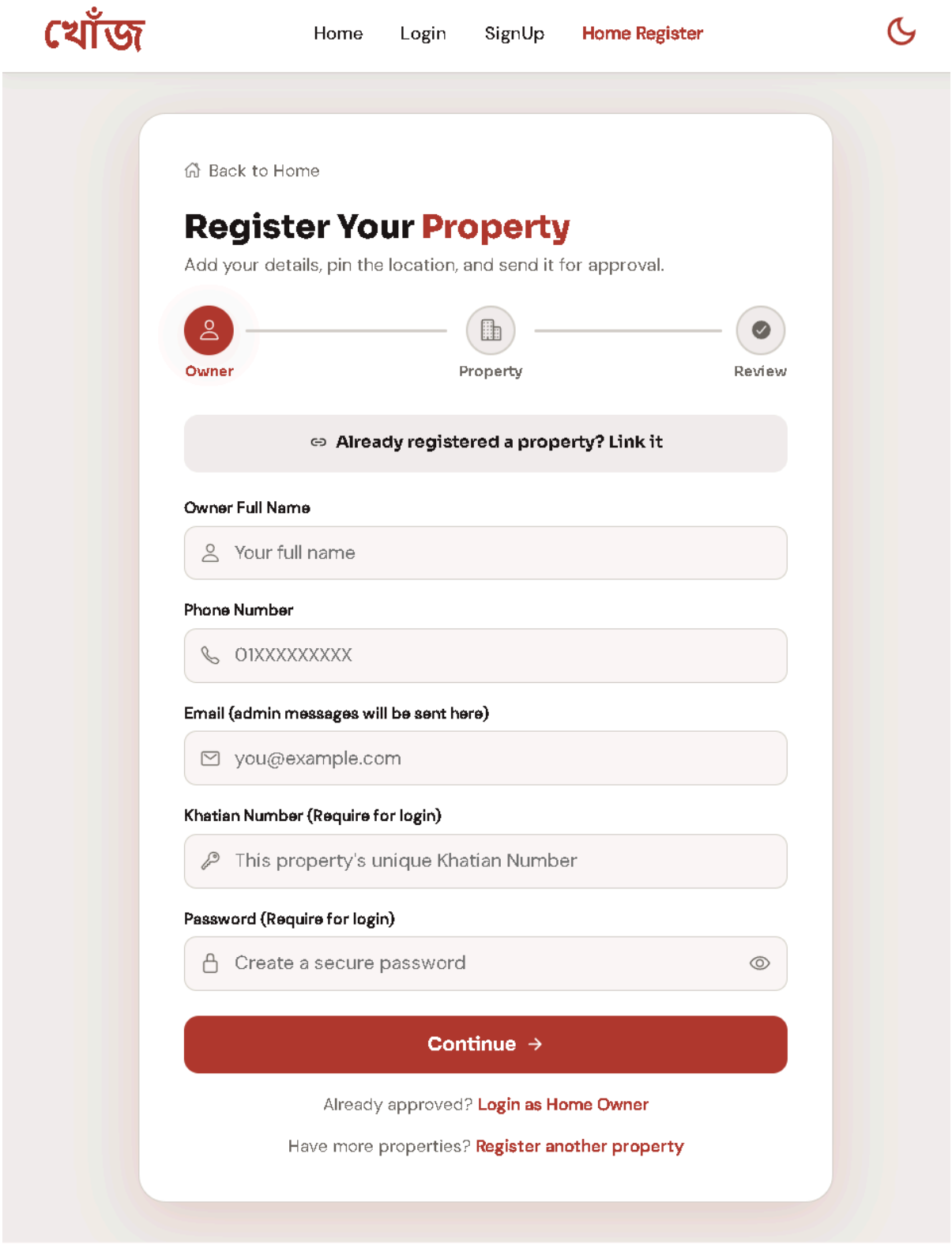
Screenshots



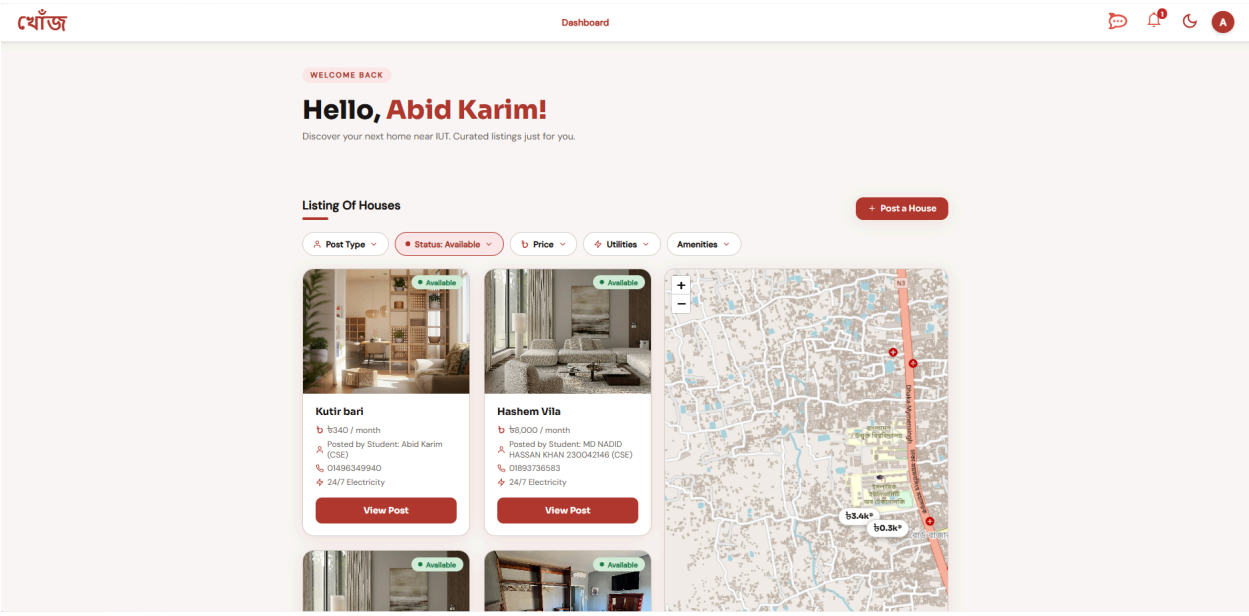
[Screenshot: landing page]



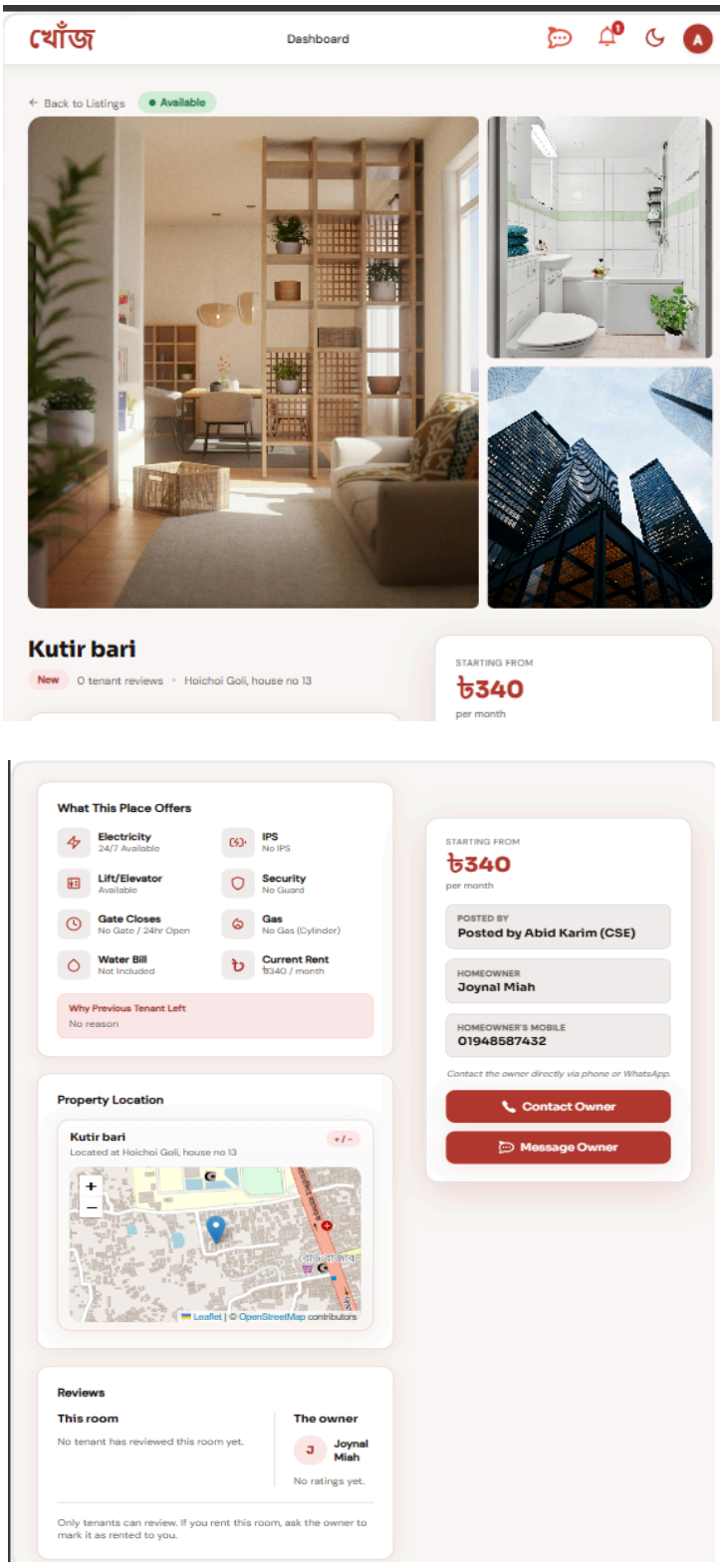
[Screenshot: student sign-up with OTP]



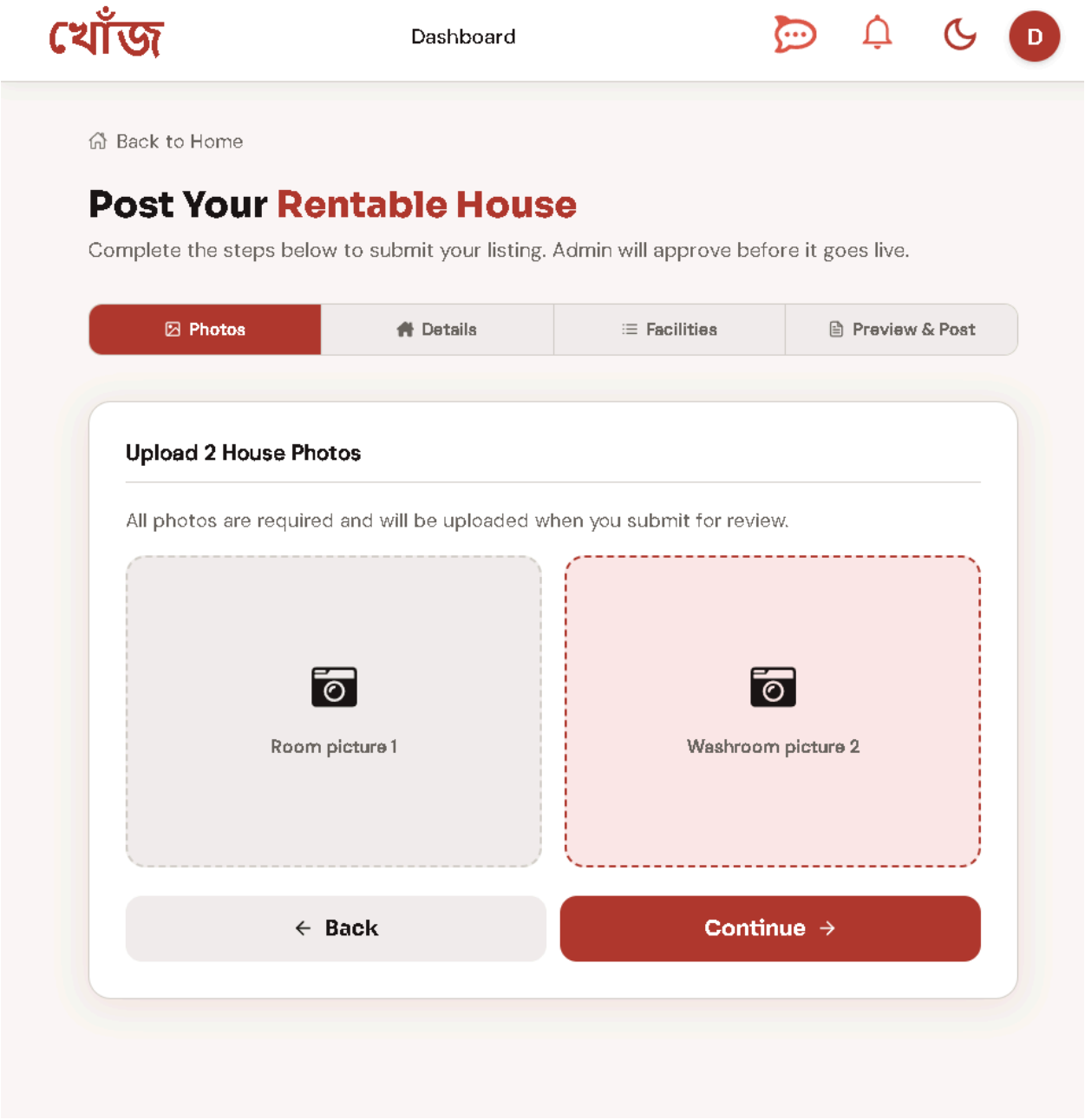
[Screenshot: owner registration]



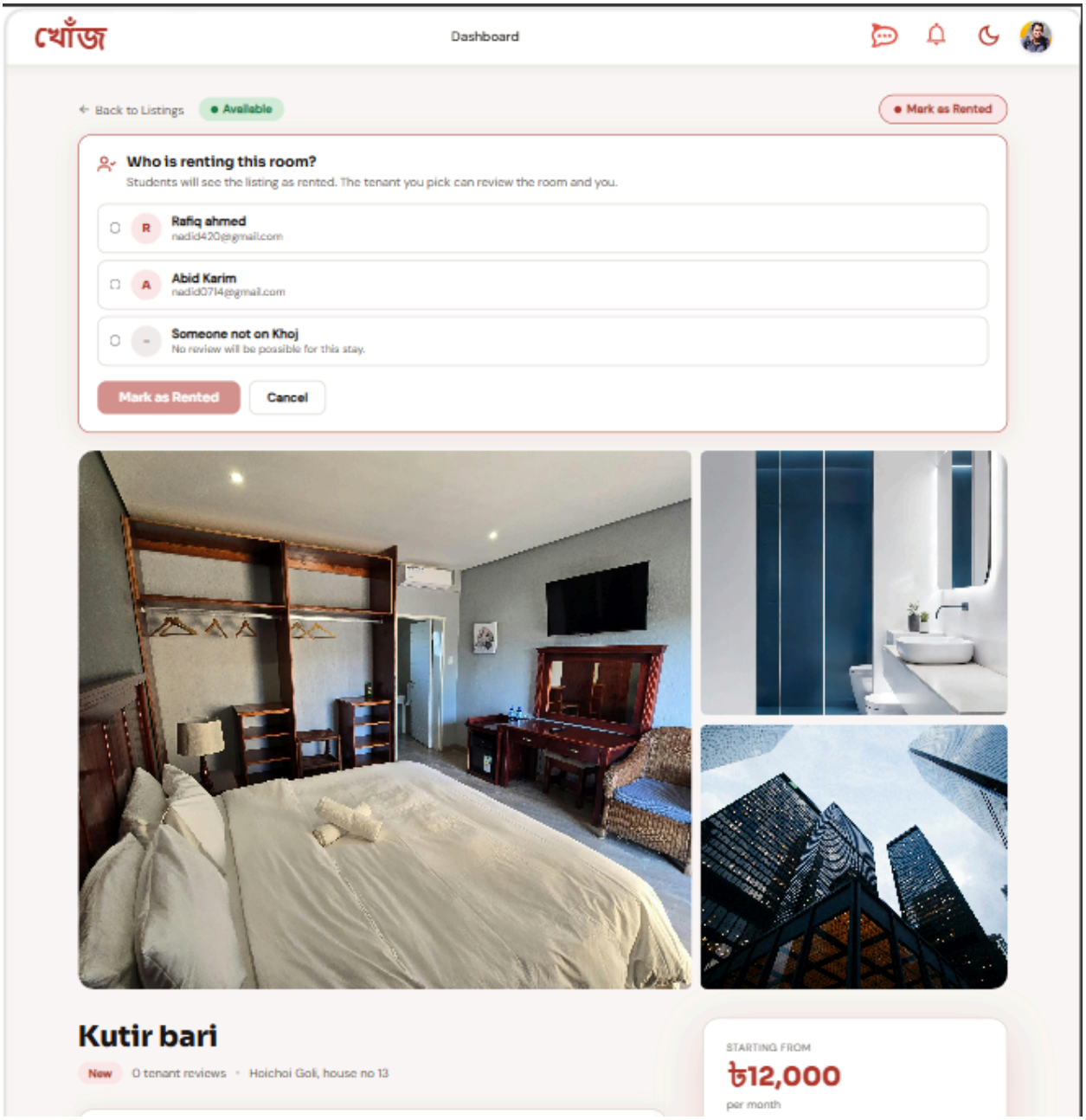
[Screenshot: dashboard with filters and map]



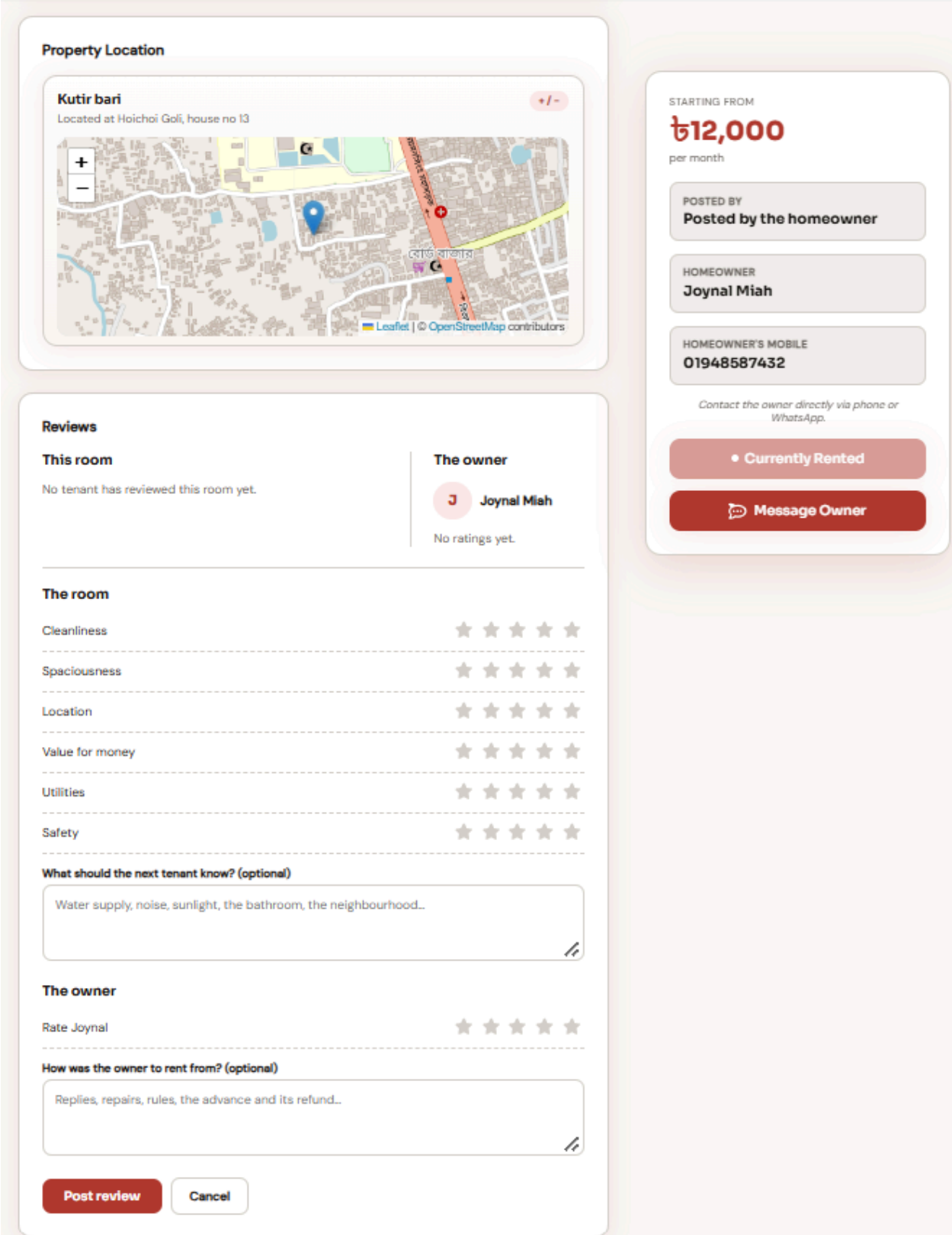
[Screenshot: listing detail with reviews]



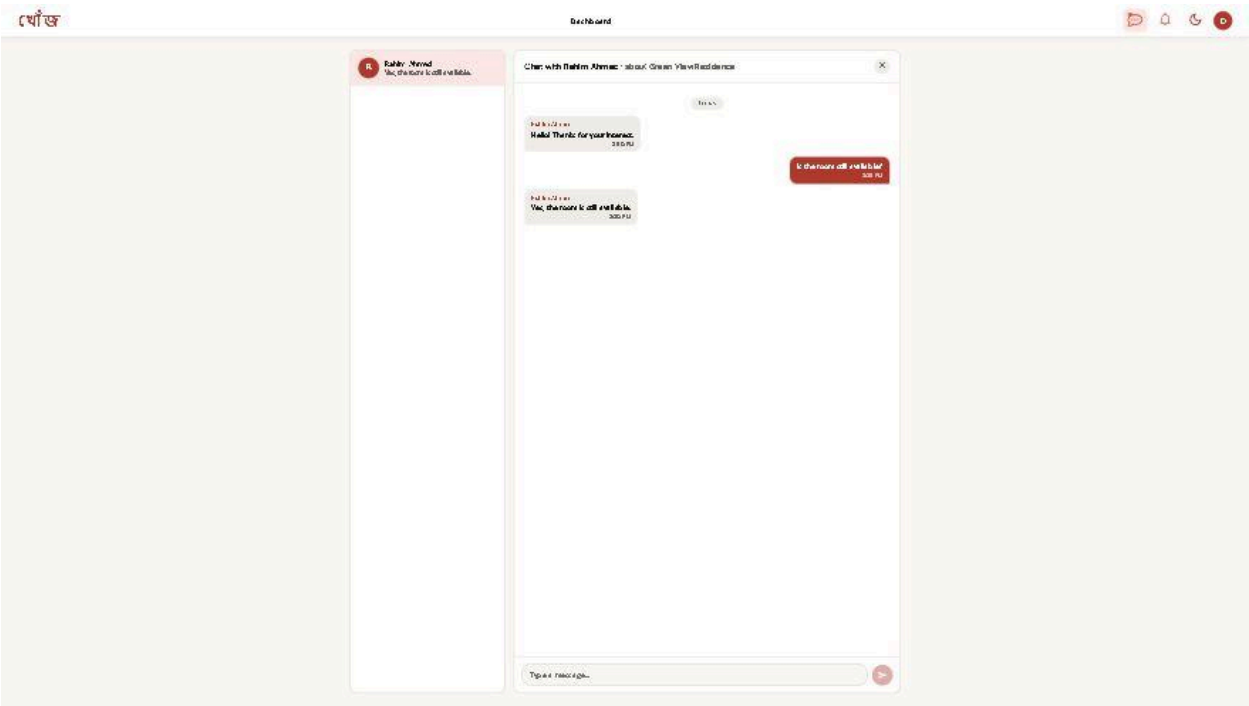
[Screenshot: create listing]



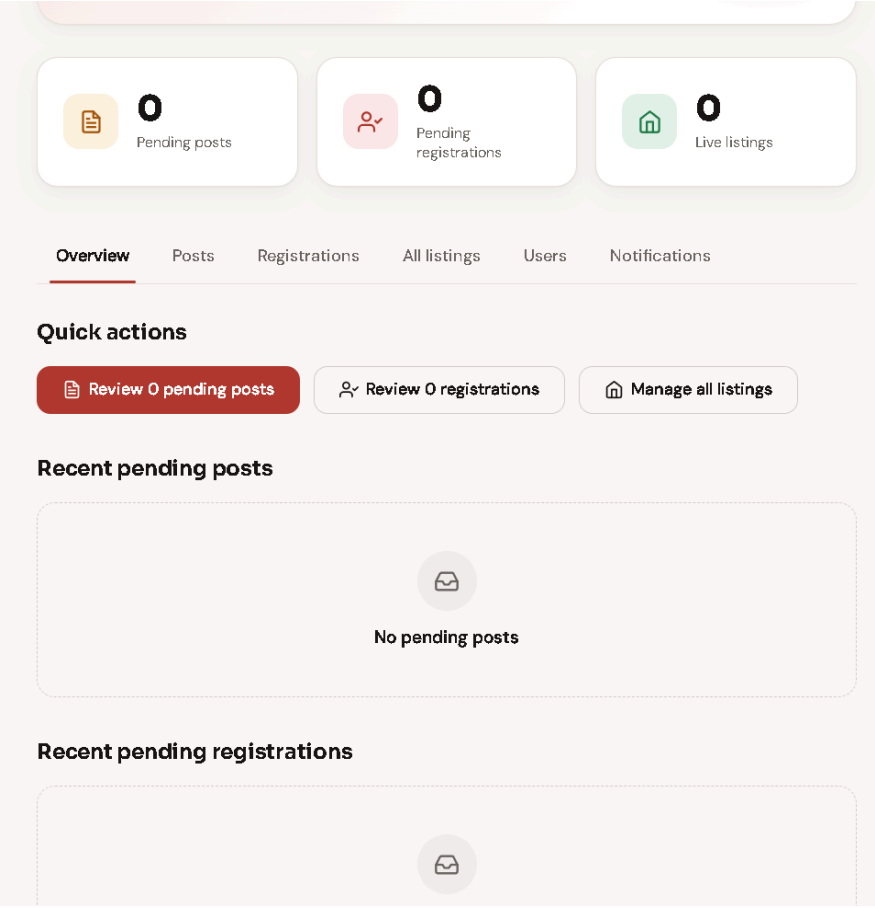
[Screenshot: tenant picker]



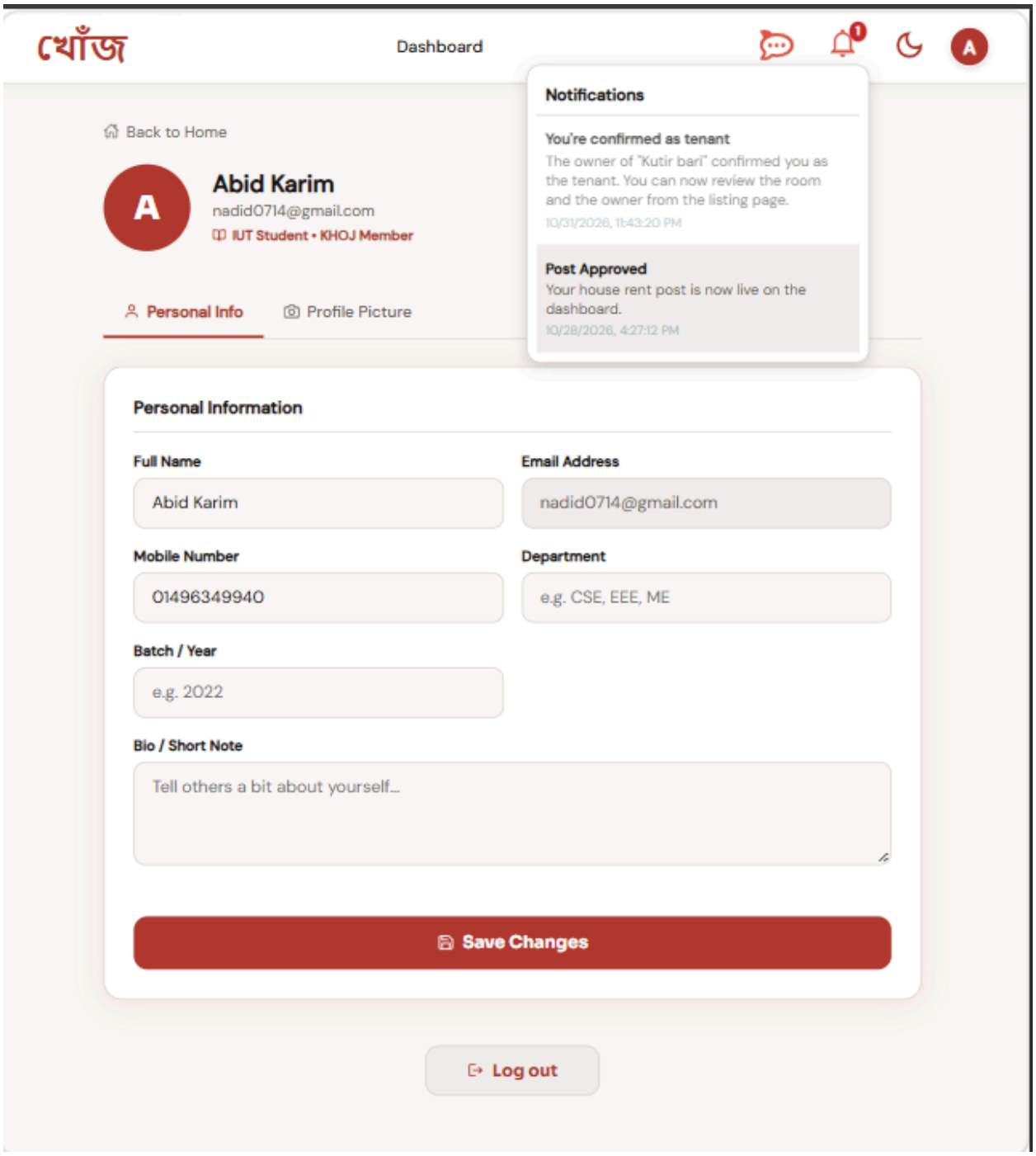
[Screenshot: review form]



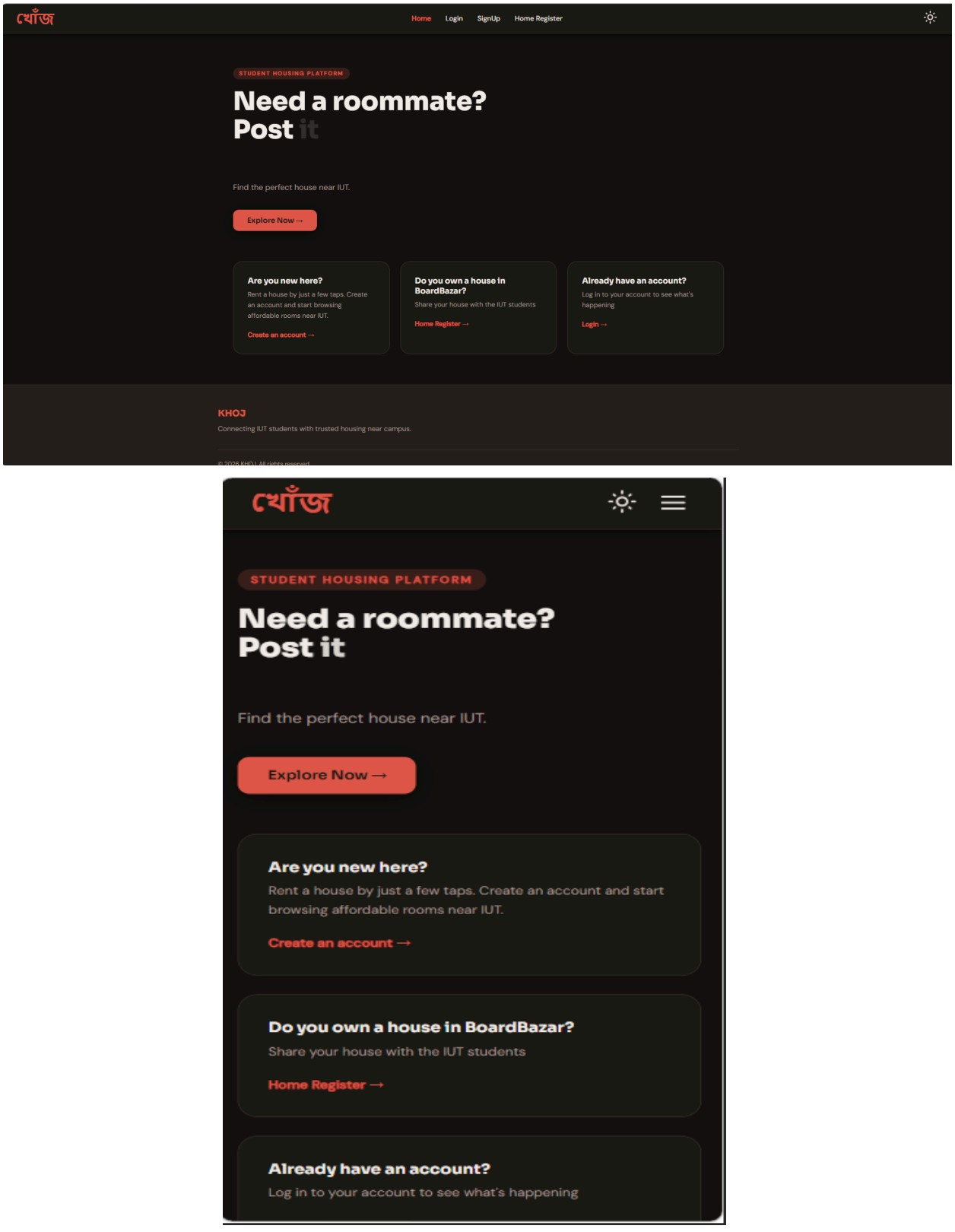
[Screenshot: chat]



[Screenshot: admin dashboard]



[Screenshot: notifications and profile]



[Screenshot: dark mode and phone view]

7. Tools, Version Control, Testing, and CI/CD

7.1 Frameworks, libraries and tools (CO4)

Tool	Used for	Alternative considered	Why chosen	Limitation
React 19	Single-page client	Plain HTML with JavaScript; Vue	Component reuse, the team's MERN skills	Create React App is no longer maintained; Vite would build faster
Express 5 on Node.js	REST API	Django, Spring Boot	Same language as the client; small and flexible	Structure depends on team discipline
MongoDB with Mongoose 9	Database	PostgreSQL (Section 5.8 B)	Flexible documents, geospatial and TTL indexes	No foreign keys; integrity enforced in code
Socket.IO 4.8	Real-time chat	Polling; raw WebSocket	Rooms, reconnection, handshake auth	Needs a shared adapter to run on several servers
Cloudinary	Image and document storage	Storing files on the server; Amazon S3	Free tier, delivery URLs, no server disk	Free-tier limits; files leave the server
Leaflet with OpenStreetMap	Maps	Google Maps	Free, no API key or billing	Tile usage policy limits heavy use
Multer	File uploads	Busboy	Simple disk storage with size and type limits	Files held on local disk until review

jsonwebtoken, bcrypt	Sessions and passwords	Server sessions; argon2	Stateless tokens; standard hashing	Token revocation needs the refresh-token check
Nodemailer with Gmail SMTP	OTP and rejection e-mails	SendGrid, Mailgun	Free with a Gmail app password	Daily sending limit of a Gmail account
Google sign-in (@react-oauth/google)	Sign-in for IUT Google accounts	Only e-mail and password	IUT accounts are Google accounts	See Section 8.3
Visual Studio Code, Git, GitHub, Postman, Nodemon, Prettier	Development	WebStorm	Free, widely used	

7.2 Version control and workflow (CO4)

The code is in a Git repository on GitHub. Each member worked on a personal branch (talha_code, talha, nadid_code, ayman_code), Nadid merged the work into main and resolved conflicts (*ef67f8f*, *c842881*, *401c7a8*). main is the source of the demonstrated system. Secrets stay out of the repository: .env files are ignored and .env.sample files list the required keys.

GitHub issues and pull requests were not used. The team coordinated directly, in meetings and through messages, and some fixes were shared as code over messages before one member committed them. This kept the team fast, but it means the commit history under-represents some members' work (Section 10.1) and leaves no written record of bugs and decisions. Using issues and pull requests, even in a small team, is the main process lesson of the project.

Feature milestones in the history:

Milestone	Commit
Initial backend and frontend structure	*b8c8d86*

Interactive property maps	*b132fe2* (ayman_code)
Dark mode	*7ac510e* (talha_code)
Full frontend and backend on main	*bee62e0*
Backend logic and API update	*bf7756f*
Real-time chat	*75f522b*
Chat and map integrated	*ecfbe50*
Cross-device access (shared CORS policy)	*ef5ee12*
Full test pass and UI update	*e3b1aa9*
Two-tier reviews, tenancy, filters	*35bac9a*
Final fixes	*604615b*, *80092ad*

7.3 Testing

Level	What was done	Result
Unit	Not automated. The client includes the React Testing Library setup, but no tests were written.	Planned
API	Early endpoints were tested with Postman. As we built the frontend so there wasn't require to check the endpoints of api	If found wrong we fix that in the code

Manual, end to end	Every feature was exercised through the interface as a student, a rank student, an owner and the admin, including both approval flows from submission to the student's view. Chat was tested between two browsers and between devices on one network.	Demonstrated live
Validation	Invalid phone numbers, coordinates, rent ranges, file types, OTP expiry and attempt limits were entered on purpose.	Rejected with field-level messages

7.4 CI/CD

There is no CI/CD pipeline yet; the system runs locally

8. Professional, Ethical, and Legal Considerations

8.1 Licences

Component	Licence	Obligation
React, Express, Mongoose, Socket.IO, Multer, jsonwebtoken, bcrypt, cors, cookie-parser, geolib, Cloudinary SDK, react-icons, @react-oauth/google, Nodemon, Prettier	MIT	Keep the copyright notice
Nodemailer	MIT-0	None
Leaflet, dotenv	BSD-2-Clause	Keep the copyright notice
react-leaflet 5	Hippocratic License 2.1	An ethics-based licence: use must respect human rights. Khoj's use is compatible, but the licence is not OSI-approved and should be reviewed before commercial use.
OpenStreetMap data and tiles	ODbL; tile usage policy	Show "© OpenStreetMap contributors"; avoid heavy tile use without a separate tile server
Cloudinary, Google sign-in, Gmail	Terms of service	Stay within free-tier and acceptable-use terms

The project's own code carries no open-source licence; all rights are reserved by the team. The copyright line on the landing page is decorative and not a registered notice.

8.2 User data and privacy

Data	Why it is collected	How it is protected
Student name, e-mail or rank, phone, department	Identity and contact	Shown only to logged-in users where needed
Owner name, phone, e-mail, Khatian number and certificate, house photo	Verification	Files stay in a private folder until approval; the Khatian number and password hash are never sent to students
Chat messages	Communication	Visible only to the two participants, checked on every message
Reviews	Trust	Linked to a confirmed tenancy

Users' passwords are hashed with bcrypt. The OTP is deleted after five minutes and after use. Rejected files are deleted from the server. The system collects no payment data.

8.3 Security measures and known gaps

Measures in place: bcrypt password hashing; JWTs in httpOnly, SameSite cookies (Secure in production); OTP with five-minute expiry, five attempts and a 60-second resend limit; file type and 10 MB size limits; filter values whitelisted and converted to strings; owner details copied from the verified record; socket handshake authentication and participant checks; one CORS policy; secrets in environment variables.

Known gaps

found by reviewing and testing the final code, with planned fixes:

Gap	Risk	Planned fix
The /api/admin/* routes do not yet use the admin-token middleware that the notification routes use	Anyone who knows the URLs could call admin actions	Apply verifyAdminToken to the whole admin router
Google sign-in trusts the e-mail sent by the browser	A forged request could log in as another IUT address	Verify the Google ID token on the server
OTP codes are stored as plain text	Low, since they expire in five minutes	Hash them like passwords

8.4 Ethical and legal issues in deployment

Fairness. Listings are shown newest first, with no paid promotion. Ratings come only from confirmed tenants, and the owner's rating combines all their properties so that a single property cannot hide a pattern.

Owner documents. Khatian certificates are personal legal documents. They are used only for verification, seen only by the admin, and should be deleted or archived under a written retention period before public deployment.

Consent and terms. A public release needs terms of use and a privacy notice that explain what is stored, who sees it and how to delete an account, in line with Bangladesh's data-protection and digital-security law at that time.

Honest reviews. A review cannot be written without a stay, which protects owners from fake reviews. Owners cannot delete reviews; a reporting process for abusive reviews is future work.

Professional conduct. No real student data was used in testing; the team used test accounts.

9. Project Management and Feasibility

9.1 Process

The team followed an agile approach with weekly meetings. User stories from the elicitation were the backlog, prioritised High, Medium and Low, and kept in the shared user-story sheet. Each session (Proposal, SRS, Design, Implementation, Final) acted as a milestone with supervisor and committee review, and the feedback became the next set of tasks. Tasks were assigned by feature in meetings and followed up through direct messages. The evidence of progress is therefore the user-story sheet, the session documents and the commit history.

9.2 Planned vs. actual milestones

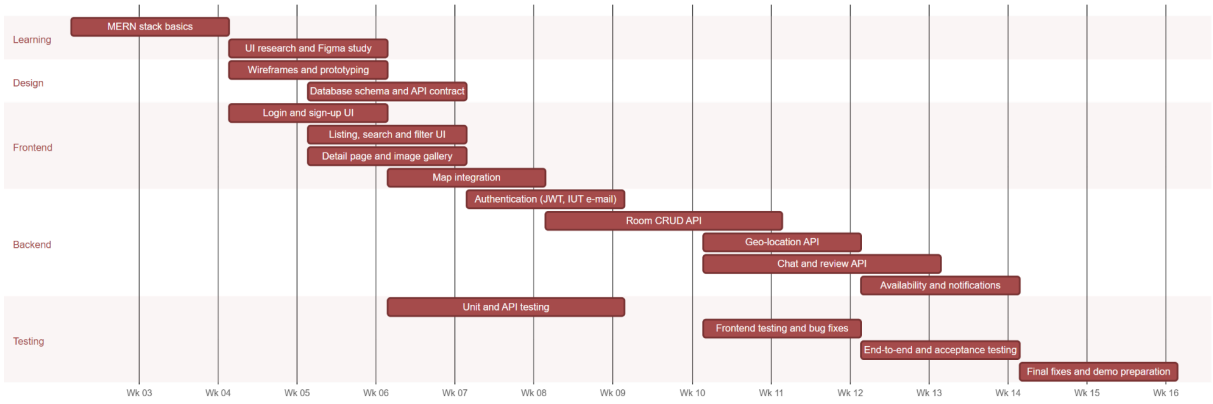


Figure 17: Planned timeline from the proposal (semester weeks)

The plan in the proposal was set in semester weeks. The actual dates below come from the commit history.

Milestone	Planned	Actual	Evidence
Learning and UI research	Weeks 3 to 6	Before coding started	Proposal and SRS documents
Design: wireframes, schema, API contract	Weeks 6 to 8	Design session; the API contract kept changing during implementation	Design report
Repository and project structure	Week 8	May 2026	*b8c8d86*
Frontend pages and map	Weeks 6 to 9	May to June 2026, refined until the end	*b132fe2*, *7ac510e*
Backend: authentication, listings, owner approval	Weeks 8 to 12	June to July 2026	*bee62e0*, *bf7756f*
Chat and notifications	Weeks 11 to 14	August 2026	*75f522b*, *ecfbe50*, *ef5ee12*
Testing and UI pass	Weeks 6 to 15 (unit, API, end to end)	October 2026; manual and API testing, no automated tests	*e3b1aa9*
Reviews, filters, final fixes	Weeks 11 to 15	Late October 2026; reviews redesigned as two tiers	*35bac9a*, *604615b*
Final presentation and live demonstration	Week 15	28 October 2026	Final presentation

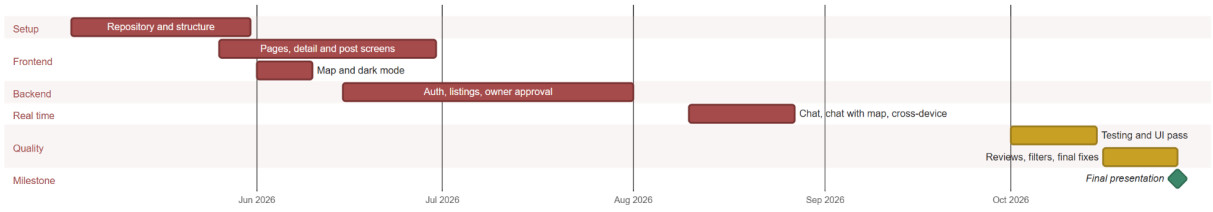


Figure 18: Actual development timeline, from the commit history

Planned vs. actual. Development took longer than the 15-week plan: active coding ran from May to October 2026. The order of work matched the plan (structure, pages and map, backend, chat, testing), but activity came in bursts around sessions rather than at a steady weekly pace, with a quiet period in September. Testing, planned to run alongside development, happened mostly at the end.

9.3 Scope and schedule changes

Change	Decision and reason	How it was decided and communicated
Moderators register owners in person → owners apply online	Supervisor feedback	Proposal revision
Moderator role merged into Admin	Too few users for two roles	Team meeting
Admission database → rank sign-up with a six-month deadline	The admission database was not available	Team meeting
Listing approval added	Keeps unverified content out of search	Team meeting
Single review → two-tier reviews from confirmed tenants	Proof of stay; room and owner judged separately	Team meeting
Stored distance and travel time removed	The straight-line values looked shorter than the real walk by road, and the team doubted that the stored gate point was exact. Without a way to measure the true walking distance, showing no number was safer than showing a wrong one. Road distance through a routing service is future work.	Team meeting

MongoDB kept instead of switching to PostgreSQL	Rewriting the data layer late would have put the remaining features at risk	Design session feedback, then a discussion with the supervisor
Payments (bKash, Nagad), recommendations, extra modules	Moved to future work	Proposal presentation listed them as additional features

Decisions taken in meetings were passed on to members through direct messages, and the affected user stories were marked in the user-story sheet.

9.4 Effort, infrastructure and cost estimates

The proposal and SRS did not include effort or cost estimates, and the team did not log working hours. The estimates below were made for this report.

Effort: We tried to work around 2 to 4 hours every weekend
Infrastructure and third-party costs: As we use Personal laptops, local MongoDB, free tiers of Cloudinary and Gmail SMTP so they not require any cost

9.5 Feasibility

Technical. Every component uses mature, documented open-source libraries the team already knew (MERN). The riskiest parts, real-time chat and file handling, were built and demonstrated.

Time. Scope was cut where needed (wishlist, reports, payment) so the core trust features could be finished within the semester.

Resources. The system runs on free tiers. One small server is enough for the students of one campus.

Operational. One admin can verify the owners around one campus. Growth to other universities would need more admins or the moderator role back.

10. Teamwork and Individual Contributions

10.1 Member Responsibilities

Member	Responsibilities	Evidence in the repository
Talha Zubayer (230042119)	Real-time chat and WebSocket layer (including cross-device access); dashboard; dark mode; rate and review (tenancy and two-tier reviews, filters)	*75f522b* chat, *ef5ee12* cross-device, *7ac510e* dark mode (talha_code, merged in *bee62e0*), *35bac9a* reviews, tenancy and filters
Ayman Rahman Bhuiyan (230042140)	Map (property maps and pins); feature testing; SRS collection	*b132fe2* interactive property maps (ayman_code); SRS documents
Md Nadid Hasan Khan (230042146)	OTP; posting listings; file upload; authentication; admin dashboard; design schema , api endpoint and integration all features	*b8c8d86* structure, *bee62e0* frontend and backend, *bf7756f* backend logic, *ecfbe50* chat and map merge, *e3b1aa9* testing and UI, *604615b* admin file preview and rejection e-mail

10.2 Coordinating interdependent modules

Shared contracts. Every reply uses ApiResponse, every error ApiError, and the frontend reads the server address from config.js. A member could build a page against another member's endpoint without asking how it replies.

Vertical slices. Each feature has its own route, controller, service and model files, so members rarely edited the same file.

Integration points. Chat had to fit the listing page ("Message owner") and the map; the two were merged in *ecfbe50*. Reviews depend on chat (tenant candidates are the students who messaged about the listing) and on the rental toggle.

Integration owner. Nadid merged components of the app and resolved the conflicts if occured during the project , so one person kept an overview of how the parts fit.

10.3 Decisions, blockers and inclusion

Situation	How it was handled
Database choice challenged in the Design session	The team weighed the expert's advice against the remaining time, discussed it with the supervisor, and kept MongoDB while recording the trade-off
Distance values looked wrong	Discussed in a team meeting; the feature was removed rather than shipped with misleading numbers
Scope larger than the time available	Features were ranked by the interview results and priorities; wishlist and fake-listing reports were dropped in a team meeting
Blocker: OTP e-mails could not be sent during development	OTP was put behind a switch so the rest of sign-up could be built, then completed with Gmail SMTP
Blocker: chat and the app were not reachable from other devices	One CORS policy shared by the REST API and the socket server
Fair participation	Features were divided by member and owned end to end. No member interact to each other component development

11. Results, Limitations, and Future Work

11.1 Status of each requirement

We have to delete some feature which priority was low due to shortage of time

Status: **Completed**, **Partial**, **Not done**.

FR	User story	Acceptance criteria (summary)	Status
FR_1	US_1 Register with IUT e-mail and OTP	@iut-dhaka.edu only; 6-digit OTP, 5-minute expiry, 5 attempts, 60-second resend	Completed
FR_1	US_2 Secure login and session	Access and refresh tokens in httpOnly cookies; wrong credentials rejected	Completed
FR_1	US_21 Register with admission rank	Unique rank; personal e-mail verified by OTP; six-month deadline; admin lists expired accounts	Completed
FR_2	US_3 Owner applies with Khatian certificate	Certificate, house photo, map pin, valid phone, unique Khatian number; pending until decided; rejection carries a reason	Completed
FR_2	US_22 Owner login with Khatian number	Khatian number and password; profile from the approved registration	Completed
FR_2	US_26 Link another property	Prefill from an earlier Khatian number; same owner group; separate login per property	Completed
FR_2	US_4 Admin verifies owner documents	Queue; view certificate and photo before deciding; approve or reject with reason; applicant informed	Completed

FR_3	US_5 Filter by rent range	Minimum and maximum; invalid ranges rejected	Completed
FR_3	US_6 Filter by distance, facilities, room type	Utilities and amenities (all selected must match), combined with rent; distance; room type	Partial (no distance filter; room type is covered by the post-type filter: roommate post or house-rent post)
FR_4	US_7 Listings on a map with distance	Pins; clicking a pin opens the card; list and map side by side; distance on each card	Partial (price-labelled pins and IUT gate marker; no distance on cards)
FR_4	US_24 Walking and riding time from the IUT gate	Distance and travel times per listing	Not done
FR_5	US_8 Available, soon vacant, rented	Status badge on each card	Partial (Available and Rented badges and a status filter; no "soon vacant")
FR_6	US_9 Wishlist	Save, list, remove	Not done (removed from scope)
FR_6	US_10 Wishlist notification	Notify when a saved room frees up	Not done (removed from scope)
FR_7	US_11 Post a room with rent, location, rules	Khatian re-check; owner details, address and map location from the verified record; rent and gate time required; room type and minimum stay	Partial (room type given by the post type; no minimum stay)
FR_7	US_12 Categorised photos	Images and PDFs up to 10 MB; photos grouped by room, bathroom, balcony, kitchen	Partial (two photos, room and washroom; no balcony or kitchen)

FR_5, FR_7	US_13 Manage availability	Only the owner toggles status; edit and delete from one dashboard	Partial (status toggle only)
FR_11	US_19 Post a room as a student	Approved property, department, phone; admin approval	Completed
FR_12	US_23 Admin approves listings	Pending until approved; photos uploaded on approval; author notified	Completed
FR_8	US_14 Chat with an owner	Start from a listing; real time; history; unread marks	Completed
FR_8	US_15 Owner replies	One inbox; listing shown; only participants see it	Completed
FR_9	US_16 Review room and owner	Confirmed tenant only; six room categories and owner rating; one per stay, editable; averages shown	Completed
FR_9	US_28 Owner confirms the tenant	Pick from students who messaged; one active tenant; tenant notified	Completed
FR_10	US_17 Report a fake listing	Report button with reason	Not done
FR_10	US_18 Admin handles reports	Report queue	Not done
FR_12	US_20 Manage users	List users, delete with reason, list expired rank accounts	Completed
FR_12	US_25 Notifications	Automatic decision notices; admin messages; bell with unread marks	Completed
FR_12	US_27 Manage profile	Students edit details and picture; owners change picture	Completed

Totals: 17 completed, 6 partial, 5 not done.

11.2 Non-functional requirements met

NFR	How it is met	Verified by
NFR_1	bcrypt hashing before save for users and owners	Database inspection
NFR_2	JWT in httpOnly, SameSite cookies; access 1 day, refresh 10 days	Browser tools
NFR_3	OTP checked again at registration and consumed	Manual test
NFR_4	Images and PDFs only, 10 MB limit	Manual test with invalid files
NFR_5	Owner details from the verified record	API test
NFR_6	Socket handshake JWT; participant check on every message	Two-browser test
NFR_7	Listing endpoints return only name and property	API response inspection
NFR_8	Stored latest message for the inbox; ratings attached in one aggregation; indexed queries	Code review
NFR_9	Responsive layout, dark mode	Phone and desktop test
NFR_10	Field-named validation messages	Manual test
NFR_11	Status changes only after all uploads succeed; temporary files deleted	Manual test
NFR_12	Layered folders	Code structure
NFR_13	.env variables listed in .env.sample	Setup on a second machine and checked in github.

11.3 Known limitations

The security gaps which are mentioned in Section 8.3.

Room ratings are returned with every listing but shown only on the listing page; dashboard cards do not show them, and map pins show "New" for every listing.

The server supports a typing indicator for chat, but the client does not use it.

No automated tests and no CI pipeline.

Runs on a local network only; not deployed.

Long page components (HomeRegister, Post, AdminDashboard) mix layout and logic.

The interface is in English only and assumes homeowners have basic digital literacy

Integrity rules live in application code because MongoDB has no foreign keys.

No issue tracker or pull requests, so decisions and bug reports are not recorded in the repository.

11.4 Future work

Close the security gaps which are mentioned in Section 8.3.

"Soon vacant" status, minimum stay, finer room types, and a distance filter.

Room ratings on dashboard cards are the typing indicator in chat.

Rent and advance payment through bKash, Nagad or Rocket, with payment history.

Recommendations from a student's searches; saved-search alerts.

Road distance and walking time from the IUT gate through a routing service, with a verified gate location.

A Bangla interface and assisted registration for homeowners with limited digital literacy.

Extension to other universities.

Deploy the app

12. Glossary and References

12.1 Glossary

Term	Meaning
API	Application programming interface: the set of web addresses the client calls to read or change data
Board Bazar	The area of Gazipur next to IUT where most students rent rooms
Cloudinary	A cloud service that stores and serves images and documents
CORS	Cross-origin resource sharing: the browser rule that decides which websites may call the server
Discriminator	A Mongoose feature that stores several related model types in one collection
DTO	Data transfer object: the shape of the data sent between two parts of the system
Gate closing time	The time the building's main gate is locked; after it, a tenant needs the owner's permission to go in or out
JWT	JSON Web Token: a signed token that proves who is logged in
Khatian	The official land record in Bangladesh that names the owners of a plot
LOC	Lines of code
Module	One JavaScript file in the code base
OTP	One-time password, sent by e-mail to confirm an address
REST	A style of web API that uses standard HTTP methods on resource addresses
Socket.IO	A library for two-way, real-time communication between browser and server

Tenancy	A stay confirmed by the owner; it allows the student to review the room and the owner
TTL index	A MongoDB index that deletes a document automatically after a set time
2dsphere index	A MongoDB index for locations on the earth's surface

12.2 References

[2] React documentation, *https://react.dev*

[3] Express documentation, *https://expressjs.com*

[4] Bikroy.com, property listings, *https://bikroy.com*

[5] bdHousing.com, *https://www.bdhousing.com*

[7] Airbnb Help Centre, "How reviews work", *https://www.airbnb.com/help*

[11] Mongoose documentation: discriminators, *https://mongoosejs.com/docs/discriminators.html*

[12] Socket.IO documentation, *https://socket.io/docs/v4/*

[14] Leaflet, *https://leafletjs.com*; OpenStreetMap tile usage policy, *https://operations.osmfoundation.org/policies/tiles/*

[15] Cloudinary documentation, *https://cloudinary.com/documentation*

[15] Nodemailer documentation, https://nodemailer.com/guides

[15] npm documentation https://www.npmjs.com/